

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA INGENIERÍA DE SOFTWARE

3 créditos



Profesor Autor:

Ing. Freddy Malla García, Mtr.

Titulaciones	Semestre
INGENIERÍA DE SOFTWARE	SEXTO

Tutorías: El profesor asignado publicará en el entorno virtual de aprendizaje online.utm.edu.ec, y sus horarios de conferencias se indicarán en la sección **CRONOGRAMA**.

PERIODO ABRIL - AGOSTO 2026



Contenido

UNIDAD V DISEÑO	1
1. DISEÑO DE SOFTWARE	1
1.1. DISEÑO EN EL CONTEXTO DE LA INGENIERÍA DE SOFTWARE	2
1.1.1. EL PROCESO DE DISEÑO	4
1.1.1.1. Lineamientos y atributos de la calidad del software	4
1.1.1.2. La evolución del diseño del software	6
1.1.2. CONCEPTOS DE DISEÑO	7
1.1.2.1. Abstracción	8
1.1.2.2. Arquitectura	8
1.1.2.3. Patrones	10
1.1.2.4. División de problemas	10
1.1.2.5. Modularidad	11
1.1.2.6. Ocultamiento de información	13
1.1.2.7. Independencia funcional	13
1.1.2.8. Refinamiento	14
1.1.2.9. Aspectos	15
1.1.2.10. Rediseño	16
1.1.2.11. Conceptos de diseño orientados a objeto	16
1.1.2.12. Clases de diseño	16
1.2. EL MODELO DEL DISEÑO	18
1.2.1. DISEÑO DE LA ARQUITECTURA:	20
1.2.1.1. Arquitectura del software	20
1.2.1.2. ¿Por qué es importante la arquitectura?	22
1.2.1.3. Géneros arquitectónicos	23
1.2.1.4. Estilos arquitectónicos	25
1.2.1.5. Breve taxonomía de estilos de arquitectura	26
1.2.1.6. Patrones arquitectónicos	29
1.2.1.7. Diseño arquitectónico	30
1.2.1.8. Representación del sistema en contexto	31
1.2.1.9. Definición de arquetipos	33
1.2.2. DISEÑO EN EL NIVEL DE COMPONENTES:	34
1.2.2.1. ¿Qué es un componente?	35
1.2.2.2. Una visión orientada a objetos	35
1.2.2.3. La visión tradicional	37
1.2.2.4. Visión relacionada con el proceso	40

1.2.2.5.	Elementos del diseño en el nivel de los componentes	41
1.2.2.6.	Diseño de componentes basados en clase.	42
1.2.2.6.1.	Principios básicos del diseño	42
1.2.2.7.	Diseño en el nivel de componentes para WEBAPPS.	45
1.2.2.8.	Diseño del contenido en el nivel de componente	45
1.2.2.9.	Diseño de las funciones en el nivel de componentes	46
1.2.2.10.	Diseño de componentes tradicionales.	47
1.2.2.11.	Desarrollo basado en componentes.	48
1.2.2.12.	Ingeniería del dominio	49
1.2.3.	DISEÑO DE LA INTERFAZ DE USUARIO:	49
1.2.3.1.	Elementos de diseño de la interfaz.....	50
1.2.3.2.	Reglas.....	53
1.2.3.3.	Dejar el control al usuario	53
1.2.3.4.	Reducir la necesidad de que el usuario memorice	54
1.2.3.5.	Hacer consistente la interfaz.....	56
1.2.3.6.	Análisis y diseño de la interfaz de usuario.	56
1.2.3.7.	Análisis de la interfaz.....	57
1.2.3.8.	Análisis del usuario	57
1.2.3.9.	Etapas del diseño de la interfaz.	58
1.2.3.10.	Diseño de una interfaz para WEBAPPS.	59
1.2.3.11.	Principios y lineamientos del diseño de la interfaz	60
1.2.3.12.	Flujo de trabajos para el diseño de la interfaz de webapp.....	65
1.3.	ELEMENTOS DEL DISEÑO DEL DESPLIEGUE	68
	BIBLIOGRAFÍA	69

Índice de tabla

Tabla 1: Atributos de calidad FURPS	6
Tabla 2: Tipos diferentes de clases de diseño	17
Tabla 3: Géneros arquitectónicos para sistemas basados en software	24
Tabla 4: Principios básicos de Diseño.....	44

Índice de Ilustración

Ilustración 1: Traducción del modelo de requerimientos al modelo de diseño	3
Ilustración 2: Modularidad y costo del software	12
Ilustración 3: Dimensiones del modelo de diseño	19
Ilustración 4: Arquitectura centrada en datos	27
Ilustración 5: Arquitectura de flujo de datos.....	27
Ilustración 6: Arquitectura de programa principal/subprograma	29
Ilustración 7: Arquitectura en capas	29
Ilustración 8: Diagrama de contexto arquitectónico 9.05	32
Ilustración 9: Diagrama de contexto arquitectónico para la función de seguridad de CasaSegura	32
Ilustración 10: Relaciones de UML para los arquetipos de la función de seguridad de CasaSegura	34
Ilustración 11: Elaboración de un componente de diseño	37
Ilustración 12: Gráfica de la estructura de un Sistema tradicional	39
Ilustración 13: Diseño en el nivel de componentes para CalcularCostoporPágina	40
Ilustración 14: Diagrama de componente UML.....	42
Ilustración 15: Seguimiento del PAC	45
Ilustración 16: Representación de la interfaz para Panel de Control	52
Ilustración 17: Mapeo de los objetivos del usuario en acciones de la interfaz.....	67
Ilustración 18: Diagrama de despliegue UML.....	69

UNIDAD V DISEÑO

1. DISEÑO DE SOFTWARE.

¿Qué es el diseño? Aquí es donde te paras sobre un pie en dos mundos: el mundo de la tecnología y el mundo de los humanos que tratan de unificarse, Vitruvio, un crítico de la arquitectura romana, dijo que los edificios que estaban bien diseñados tienen fuerza, función y belleza. Lo mismo ocurre con un buen software (Pressman, 2010).

Resistencia: un programa no debe contener errores que impidan su funcionamiento.

Funcionalidad: un software debe ser adecuado para los fines que busca.

Belleza: La experiencia de usar el software debe ser placentera. Este es el comienzo de la teoría del diseño de software (Pressman, 2010).

El propósito del diseño es crear una forma o representación que tenga fuerza, funcionalidad y belleza, para lograr esto, debemos practicar la diversificación y luego la convergencia Belady [Bel81] afirma que “La diversificación es la adquisición de una combinación de alternativas, materia prima del diseño: componentes, soluciones con los componentes y conocimiento todo en catálogos, guías y en mente (Pressman, 2010).

Después de compilar este grupo de información diversa, artículos se encuentran los requisitos que se definen por tecnología y modelo analítico, cuando esto sucede, se evalúan las alternativas, algunas de las cuales son desatendidas, convergen con "una composición específica de los componentes y con ellos, en la creación del producto final" [Bel81]. Diversidad y convergencia combinan intuición y juicio basados en la experiencia en la construcción de entidades similares, un conjunto de principios de experiencia guiando el desarrollo del modelo, un conjunto de criterios de evaluación de la calidad y proceso iterativo que finalmente culmina en la entrega del diseño final (Pressman, 2010).

El diseño de software cambia constantemente a medida que evolucionan nuevos métodos, emergen mejores análisis y se obtiene una comprensión más amplia, incluso hoy en día, mayor parte de las técnicas de diseño de software carecen de profundidad, flexibilidad y naturaleza cuantitativa, a menudo asociado con disciplinas de diseño de ingeniería más tradicionales. Sin embargo, si hay formas de diseñar software, existen estándares de diseño con calificaciones de calidad y diseños aplicados (Pressman, 2010).

1.1. DISEÑO EN EL CONTEXTO DE LA INGENIERÍA DE SOFTWARE.

El diseño de software pertenece al campo técnico de la ingeniería de software y se aplica sin importar el modelo del proceso que se utilice. El diseño del programa comienza una vez después de analizar y modelar los requisitos, este es el último trabajo de ingeniería de software en las actividades de modelado y preparación para la fase de construcción (construcción y verificación de código). Cada elemento del formulario de requisitos proporciona información necesaria para crear los cuatro modelos de diseño necesarios para la especificación completa de diseños (Pressman, 2010).

La ilustración 1 muestra el flujo de información durante el proceso de diseño del sistema. El trabajo de diseño está impulsado por el modelo de requisitos, que representa los elementos basados en texto, basados en clases, orientados al flujo y de comportamiento. El empleo de la notación y de los métodos de diseño que produce diseños de los datos o clases, de la arquitectura, de la interfaz y de los componentes (Pressman, 2010).

El diseño de datos o clases transforma los modelos de clases en aplicaciones clases de diseño y estructuras de datos necesarias para implementar el programa. Los objetos y las relaciones se definen en un esquema CRC y contenido de datos detallado o descritos por atributos de clase y otros tipos de notación proporcionan la base para el diseño de los datos. Parte del diseño de la clase se puede realizar junto con el diseño de la arquitectura del sistema. Se produce un diseño de capas más detallado al diseñar cada componente del software (Pressman, 2010).

El diseño arquitectónico determina la relación entre los elementos principales de la estructura del Software, estilos de diseño arquitectónico y patrones que se pueden usar para cumplir con los requisitos y restricciones definidos por el sistema y que afectan la forma en que se implementa la arquitectura [Sha96]. Una expresión de diseño de la arquitectura – marco para sistema informático - Tomado del formulario de requisitos (Pressman, 2010).

El diseño de interfaz describe cómo el software se comunica con los sistemas que interactúan con él y las personas que lo usan. La interfaz consiste en un flujo de información (como un dato o control) y un tipo específico de comportamiento (Pressman, 2010).

Después, los modelos de escenarios de uso y comportamiento proporcionan información muy necesaria para el diseño de la interfaz. El diseño a nivel de componentes transforma los

elementos estructurales de la arquitectura del programa que describe sus componentes desde un punto de vista procedimental (Pressman, 2010).

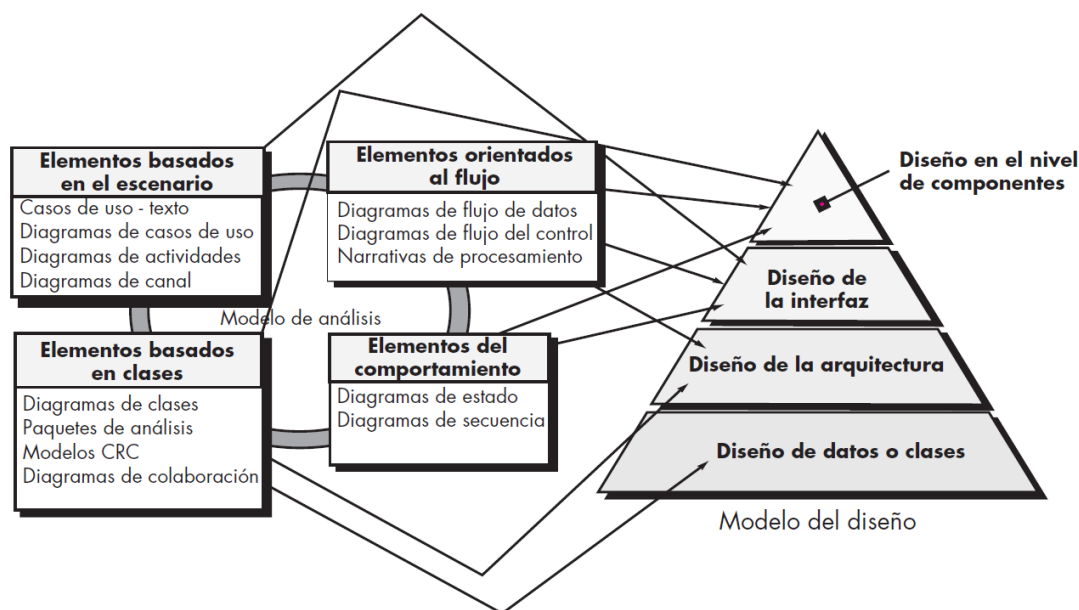


Ilustración 1: Traducción del modelo de requerimientos al modelo de diseño

Fuente: (Pressman, 2010).

La información obtenida de modelos de clase, subprocesso y comportamiento que actúan como base de diseño de componentes. Se toman decisiones durante el proceso de diseño que finalmente afectarán el éxito de la construcción del programa, e igual de importante, la facilidad de mantenimiento en el programa. Pero, ¿por qué es importante el diseño? La importancia del diseño de software se puede resumir, en una palabra: calidad (Pressman, 2010).

El diseño del sitio web ofrece calidad en ingeniería de software. Da representaciones del software que pueden evaluarse en su calidad. Esta es la única forma de traducir el producto final o pedido requerido por el participante. Es la base de toda Ingeniería de software y actividades de soporte relacionadas. Sin diseño tenemos riesgo de que un sistema inestable se bloquee al realizar pequeños cambios o es difícil o imposible evaluar su calidad hasta que es demasiado tarde al final del proceso del software, cuando no queda mucho tiempo y ya se ha utilizado mucho dinero (Pressman, 2010).

1.1.1. EL PROCESO DE DISEÑO

El diseño de software es un proceso iterativo a través del cual se traducen los requisitos en el "plano" para la creación de software. En primer lugar, el plano muestra una descripción general del software. Es decir, el diseño se expresa con un alto nivel de abstracción, ya que se dirige el propósito específico del sistema y requisitos de datos más detallados, actividades y comportamiento (Pressman, 2010).

A medida que avanza el proceso de iteración del diseño, otras mejoras conducen a un menor nivel de abstracción. También se puede rastrear si se cumple con los requisitos, pero la conexión es más precisa (Pressman, 2010).

1.1.1.1. *Lineamientos y atributos de la calidad del software*

A través del proceso de diseño, se evalúa su calidad por medio de una serie de pruebas técnicas. McGlaughlin [McG91] sugiere tres características como guía para evaluar un buen diseño:

- Debe cumplir con todos los requisitos expresamente establecidos en su modelo de requerimiento solicitar y cumplir todos los requisitos implícitos que desee el participante.
- Debe ser una guía fácil de leer y comprender para los creadores de código y para aquellos que lo prueban y apoyan el seguimiento.
- Debe proporcionar una imagen completa del programa y los campos de dirección para datos, funcionalidad y comportamiento desde una perspectiva de implementación.

De hecho, cada una de estas características es un objetivo del proceso de diseño, pero ¿cómo se alcanza? (Pressman, 2010).

Lineamientos de la calidad. Para evaluar la calidad del diseño, usted y otros miembros del equipo del programa deben establecer estándares técnicos para un buen diseño (Pressman, 2010).

En este momento, considere los siguientes lineamientos para el diseño:

1 Debe tener una estructura 1) usando estilos o patrones arquitectónicos reconocibles, 2) compuesto por componentes con buenas características de diseño, y 3) implementado de manera escalable, de manera que sea fácil de implementar y probar (Pressman, 2010).

2 Deben ser modular, es decir, el programa debe dividirse lógicamente en elementos o subsistemas.

3 Debe contener diferentes representaciones de datos, arquitectura, interfaces y componentes.

4 Debería dar como resultado la estructura de datos adecuada para las clases que se implementarán y fuera de los modelos de datos reconocibles.

5 Debe dar como resultado componentes con características funcionales independientes.

6. Debe conducir a interfaces que reduzcan la complejidad de las comunicaciones entre componentes y el ambiente externo.

7 Debe recopilarse mediante un método iterativo que se base en la información obtenida durante el análisis de los requisitos del software.

8 Debe estar representado por una notación que transmita efectivamente su significado.

Estas pautas de diseño no se lograron por casualidad. Se logra mediante la aplicación de principios básicos de diseño, metodología y pruebas sistemáticas (Pressman, 2010).

Atributos de la calidad. Hewlett-Packard [Gra87] ha desarrollado un conjunto de atributos de la calidad del software por el cual se acuñó el acrónimo FURPS: funcionalidad, usabilidad, confiabilidad, rendimiento y mantenibilidad. Los atributos de calidad de FURPS representan el objetivo de todo diseño de todos los programas: (Pressman, 2010).

Atributos de calidad FURPS	
Funcionalidad	Se califica de acuerdo con el conjunto de características y capacidades del programa, la generalidad de las funciones que se entregan y la seguridad general del sistema
Usabilidad	Se evalúa tomando en cuenta factores humanos, la estética general, la consistencia y la documentación
Confiabilidad	Se evalúa con la medición de la frecuencia y gravedad de las fallas, la exactitud de los resultados que salen, el tiempo medio para que ocurra una falla (TMPF), la capacidad de recuperación ante ésta y lo predecible del programa.
Rendimiento	Se mide con base en la velocidad de procesamiento, el tiempo de respuesta, el uso de recursos, el conjunto y la eficiencia.
Mantenibilidad	Combina la capacidad del programa para ser ampliable (extensibilidad), adaptable y servicial (estos tres atributos

	se denotan con un término más común: mantenibilidad), y además que pueda probarse, ser compatible y configurable (capacidad de organizar y controlar los elementos de la configuración del software, y que cuente con la facilidad para instalarse en el sistema y para que se detecten los problemas
--	---

Tabla 1: Atributos de calidad FURPS

Fuente: (Pressman, 2010).

No todos los atributos de calidad del software tienen el mismo peso al diseñar. Una aplicación tal vez se aboque a lo funcional con énfasis en la seguridad. Otros pueden buscar rendimiento usando mira la velocidad de procesamiento. En el tercero, se busca la confiabilidad sin importar la ponderación, es importante tener en cuenta que estos atributos de calidad deben considerarse al comienzo del diseño, no cuando se completa el diseño y la construcción (Pressman, 2010).

1.1.1.2. La evolución del diseño del software

La evolución del diseño de software es un proceso continuo que abarca casi seis décadas. El trabajo de diseño inicial se centró en los estándares estándar de desarrollo de software [Den73] y las formas de mejorar la arquitectura del software utilizando un enfoque de software de arriba abajo [Wir71]. Los aspectos procedimentales del diseño evolucionaron hacia una filosofía conocida como programación estructurada [Dah72], [Mil72]. Se recomiendan las siguientes acciones métodos para traducir [Ste74] flujos de datos o estructuras de datos (por ejemplo, [Jac75], [War74]) para definir el diseño. Nuevos enfoques (p. ej., [Jac92], [Gam95]) propone un enfoque orientado a objeto para diseñar derivaciones (Pressman, 2010).

Al desarrollar un programa, la atención se centra en su arquitectura [Kru06] y sus modelos. El diseño se puede utilizar para implementar estructuras y niveles más bajos de abstracción de diseño (por ejemplo, [Hol06], [Sha05]). Se da mayor importancia a los métodos orientados a aspectos (por ejemplo, [Cla05], [Jac04]), desarrollo basado en modelos [Sch06] y Experimental [Ast04], enfocados a lograr modularidad, eficiencia y la estructura arquitectónica de los diseños creados (Pressman, 2010).

Muchos métodos de diseño diferentes se aplican en la industria del software además de los ya mencionados. Al igual que los métodos analíticos, cada método de diseño de software introduce heurística y notación única, así como un punto de vista sobre lo que caracteriza a la calidad en el diseño. Sin embargo, todos estos métodos tienen algunas características en común: 1) un mecanismo para convertir un modelo de requisitos en una representación de diseño, 2) una notación para representar componentes funcionales y sus interfaces, 3) un experimento para mejorar y generar puntajes y 4) una guía de evaluación de la calidad (Pressman, 2010).

Independientemente del método de diseño utilizado, se debe aplicar un conjunto de conceptos base para el diseño a nivel de datos, arquitectura, interfaz y componentes (Pressman, 2010).

1.1.2. CONCEPTOS DE DISEÑO

En la historia de la ingeniería de software, se han desarrollado una variedad de conceptos necesarios para su diseño. Aunque el nivel de interés se ha diversificado a lo largo de los años en cada concepto, todos han resistido la prueba del tiempo. Cada uno le da al diseñador del software la base sobre la que se pueden aplicar métodos de diseño complejos. Todos ayudan a responder las siguientes preguntas: (Pressman, 2010).

- ¿Cuáles son los criterios utilizados para dividir el software en sus componentes individuales?
- ¿Cómo extrae los detalles del trabajo o la estructura de datos de la representación conceptual de software?
- ¿Cuáles son los estándares unificados que determinan la calidad técnica del diseño de un proyecto?

M. Jackson [Jack 75] dijo: “El comienzo de la sabiduría [para un ingeniero de software] es comprender la diferencia entre ejecutar un programa y hacerlo funcionar.” Proporciona los conceptos básicos de diseño de software estructural necesarios para hacerlo bien (Pressman, 2010).

1.1.2.1. Abstracción

Al considerar una solución estándar para cualquier problema, se pueden preguntar muchos niveles de abstracción. Al más alto nivel, la solución se define en términos brutos con el uso del lenguaje del entorno del problema. En un nivel más bajo de abstracción se da la descripción más detallada de la solución. Terminología adecuada orientada a problemas con el cual tiene como objetivo ponerlo en práctica, tratando de encontrar una solución (Pressman, 2010).

Finalmente, en el nivel más bajo de abstracción, se propone la solución, para que pueda ser implementada directamente. Con el desarrollo de diferentes niveles de abstracción, se está trabajando para crear niveles de abstracción tanto acciones como datos. La abstracción de procedimientos es una serie de instrucciones con funcionalidad específica y limitada (Pressman, 2010).

El nombre de abstracción de procedimiento se refiere a estas funciones, pero se omiten los detalles. por ejemplo, sería la palabra abrir, en el caso de una puerta. Abrir implica una secuencia larga de pasos del procedimiento (caminar hacia la puerta, llegar y tomar el picaporte, girar éste y jalar la puerta, retroceder para que la puerta se abra, etcétera) (Pressman, 2010).

La parte de abstracción de datos es un conjunto de datos con nombre que describe un objeto de datos. En el contexto de la abstracción procesal abrir la abstracción se puede definir los datos y se llama puerta. Como cualquier objeto de datos, la abstracción de datos para la puerta reunirá un conjunto de atributos que describen la puerta (tipo, dirección de apertura, mecanismo de apertura, peso, dimensiones, etc.). Concluimos que la abstracción procedimental abrir usaría la información proporcionada en las propiedades de extracción de datos puerta (Pressman, 2010).

1.1.2.2. Arquitectura

La arquitectura de software se refiere a “la estructura general de un programa y la forma en que se hace y da integración conceptual al sistema [Sha95a]. En su forma más simple, la arquitectura es la estructura organizativa de los componentes del programa (módulo), la forma en que interactúan con él y con las estructuras de datos que utilizan. Sin embargo, en

un sentido más amplio, los componentes se generalizan para representar los elementos de un sistema grande y sus interacciones (Pressman, 2010).

Uno de los objetivos del diseño del programa es tener una arquitectura aproximada del sistema. Este es el marco dentro del cual se llevan a cabo actividades de diseño más detalladas. Un conjunto de modelos arquitectónicos que permiten a los ingenieros de software resolverlos los problemas generales de diseño. Shaw y Garlan [Sha95a] describen un conjunto de rasgos que deben identificarse como parte del diseño de la arquitectura: (Pressman, 2010).

Propiedades de la estructura. Este aspecto de la representación del diseño arquitectónico define los componentes de un sistema (módulos, objetos, filtros, etc.) y la manera en la que están agrupados e interactúan unos con otros. Por ejemplo, los objetos se agrupan para encapsular tanto los datos como procedimiento que manipula e interactúa a través de métodos de invocación (Pressman, 2010).

Propiedades no funcionales. La descripción del diseño arquitectónico debe explicar cómo la arquitectura de diseño cumple con los requisitos de rendimiento, capacidad y confiabilidad, seguridad y adaptabilidad, así como otras características del sistema (Pressman, 2010).

Familia de sistemas relacionados. El diseño arquitectónico debe basarse en patrones repetibles, esto se ve a menudo en el diseño de familias de sistemas analógicos. Básicamente, el diseño debe ser capaz de reutilizar bloques de construcción básicos (Pressman, 2010).

Dada la especificación de estas características, el diseño arquitectónico está representado por el uso de uno o más modelos diferentes [Gar95]. El modelo estructural representa la arquitectura como un conjunto organizado de componentes del programa (Pressman, 2010).

Los modelos de marco aumentan el nivel de abstracción del diseño, al intentar aprender sobre patrones de diseño arquitectónico repetibles se encuentran en tipos similares de aplicaciones. Los modelos dinámicos se ocupan de los aspectos estructurales de la arquitectura del software y muestran cómo la estructura o composición del sistema cambia de acuerdo con eventos externos (Pressman, 2010).

Los modelos de Procesos se enfocan en diseñar el proceso comercial o técnico que el sistema necesita respaldar. Finalmente, los modelos funcionales se utilizan para representar jerarquías funcionales (Pressman, 2010).

Para representar estos modelos se han desarrollado varios lenguajes de descripción arquitectónica (LDA) [Sha95b]. Aunque se han propuesto muchas AVD diferentes, la mayoría tiene un mecanismo para describir los componentes del sistema y cómo están conectados (Pressman, 2010).

Cabe señalar que existe cierta controversia sobre el papel de la arquitectura en el diseño. Algunos académicos argumentan que la adquisición de ingeniería de software debe ser separada desde el diseño y lo que sucede entre los requisitos de ingeniería y los procedimientos de diseño más común. Otros creen que definir la estructura es una parte integral del proceso de diseño (Pressman, 2010).

1.1.2.3. Patrones

Brad Appleton define un patrón de diseño como: "Es un compuesto por opiniones privadas que contienen la esencia de una solución comprobada a un problema repetido en el contexto de la demanda competitiva" [App00]. El patrón de diseño describe la estructura de diseño que resuelve un problema del diseño específico en ciertos contextos y entre "fuerzas" influyen en el método donde se aplica y utilizando el patrón mencionado. El propósito de cada patrón de diseño es proporcionar una descripción que permita al diseñador determinar: 1) si el patrón es aplicable al trabajo en cuestión, 2) si se puede reutilizar (lo que ahorra tiempo de diseño) y 3) si sirve de guía para desarrollar otro modelo en función o en estructura (Pressman, 2010).

1.1.2.4. División de problemas

La división de problemas es un concepto de diseño que demuestra que cualquier problema complejo se puede manipular más fácilmente si se subdivide en elementos susceptibles de resolverse u optimizarse de manera independiente. El problema es una característica o comportamiento especificado en el formulario de requisitos de software (Pressman, 2010).

Al dividir el problema en sus piezas más pequeñas es más fáciles de manejar, y requiere menos esfuerzo y tiempo para resolverlo. Si para dos problemas, p1 y p2, la complejidad observada de p1 es mayor que la complejidad percibida para p2, concluimos que el esfuerzo requerido para resolver p1 es mayor que el requerido para resolver p2 (Pressman, 2010).

En el caso general, este resultado es intuitivamente obvio. Lleva más tiempo resolver un problema difícil. También concluyó que cuando se combinan dos problemas, la complejidad a menudo percibida es mayor que la suma de las complejidades calculadas por separado. Esto lleva a la estrategia de divides y vencerás, porque es más fácil resolver un problema complejo si se divide en partes relajante. Esto tiene implicaciones importantes para el modularidad del software. La división de problemas se manifiesta en otros conceptos de diseño relacionados: modularidad, aspectos, independencia de funcionamiento y mejora (Pressman, 2010).

1.1.2.5. Modularidad

La modularidad es la manifestación más común de la segmentación de problemas. El programa se divide en componentes individuales distintos y abordables por separado, a veces llamados Módulos, que integran para cumplir con los requisitos del problema. Se ha dicho que “La modularidad es la única característica del software que le permite que un programa sea gestionable intelectualmente” [Mye78]. EL programa monolítico (programa grande consta de un solo módulo) no es fácil de entender para un ingeniero de software (Pressman, 2010).

El número de trayectoria de control, rango de referencia, número de variable y complejidad general, esto hará que la comprensión sea casi imposible. El diseño debe descomponerse en varias unidades con la esperanza de que sea más fácil de entender y, por lo tanto, reduciendo el costo requerido para el desarrollo de software. En vista de la división de problemas, se puede concluir que, si el programa está dividido indefinidamente, y el esfuerzo requerido para desarrollarlo será mínimo, desafortunadamente, hay otras fuerzas trabajando para que esto suceda, la conclusión es (desafortunadamente) inválida (Pressman, 2010).

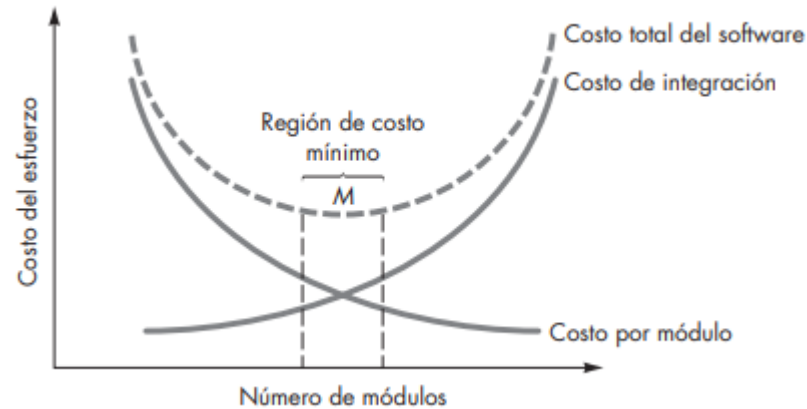


Ilustración 2: Modularidad y costo del software

Fuente: (Pressman, 2010).

De acuerdo con la ilustración 2, el esfuerzo (Costo) de desarrollar un programa individual disminuye a medida que aumenta el número total de unidades. Con el mismo conjunto de requisitos, tener más unidades significa tamaños impares, sin embargo, a medida que aumenta el número de unidades, el esfuerzo (costo) asociado a su integración también aumenta. Estas características conducen a la curva de costo total es como se muestra en la ilustración 2. Hay un cierto número, M, de módulos que ahorrará el costo mínimo de desarrollo, pero las herramientas necesarias no están disponibles para predecir M correctamente (Pressman, 2010).

Las curvas de la ilustración 2, son una guía útil cuando se considera la modularidad. Se deben tomar módulos, pero asegúrese de permanecer en la cercanía de M. Evite hacer menos o más unidades. Pero, ¿cómo sabes qué tan cerca está M? ¿Cómo debe el software ser modular? Las respuestas a estas preguntas requieren comprensión (Pressman, 2010).

El diseño (y el programa resultante) debe hacerse mediante módulos, para que el desarrollo se puede planificar más fácilmente y los pasos se pueden identificar y desarrollar programas, se facilitan los cambios, pruebas y depuración. Las operaciones más eficientes y el mantenimiento a largo plazo se llevan a cabo sin compromiso colaterales de importancia (Pressman, 2010).

1.1.2.6. Ocultamiento de información

El concepto de modularidad lleva a una pregunta fundamental: “¿Cómo se degrada la solución de programa para el mejor conjunto de unidades?” “El Principio de Confidencialidad de la información establece que los módulos "presentan decisiones de diseño implícitas" (cada uno) de los demás”, es decir, los módulos deben estar definidos y diseñados, para que no se pueda acceder a la información (algoritmos y datos) del módulo (Pressman, 2010).

El ocultamiento implica que la unidad efectiva se logra definiendo un conjunto de módulos, pero solo intercambia la información necesaria para realizar el trabajo de software (Pressman, 2010).

La abstracción ayuda a identificar entidades procesales (o informativas) que creen un programa. El ocultamiento define y hace cumplir las restricciones de acceso tanto los detalles de procedimiento como las estructuras de datos locales utilizadas por el modulo [Ross 75] (Pressman, 2010).

El uso del ocultamiento como estándar de diseño para sistemas modulares proporciona el máximo beneficio cuando se necesitan cambios durante las pruebas y luego durante el mantenimiento del programa. Dado que la mayoría de los datos y los detalles de procedimiento están ocultos de otras partes del programa, menos probable que los errores involuntarios cometidos durante el proceso de edición se transmiten a diferentes sitios web dentro del programa (Pressman, 2010).

1.1.2.7. Independencia funcional

El concepto de independencia funcional es una consecuencia directa de la separación de problemas y de conceptos abstractos y ocultación de información. En artículos importantes sobre diseño, Wirth [Wir71] y Parnas [Par72] se refieren a técnicas de optimización que benefician la independencia modulara (Pressman, 2010).

La independencia funcional se obtuvo desarrollando módulos con funciones de “miopes” tienen una "aversión" a la interacción excesiva con otras unidades. En otras palabras, debe diseñar programas para que cada módulo aborde un subconjunto específico de requisitos y tenga una interfaz simple cuando se ve desde otras partes de la estructura del programa. Tiene sentido preguntarse por qué la independencia es tan importante (Pressman, 2010).

Software eficientemente modular, es decir, con módulos independientes, más fácil desarrollo porque se deshabilitan sus funciones y se simplifican las interfaces (el desarrollo realizado por un grupo, las ramificaciones deben ser consideradas). Los módulos independientes son fáciles de mantener (y testear) debido a los efectos secundarios que provocan el diseño o modificación del código es limitado, lo que reduce la posibilidad de propagación de errores, y es posible conseguir unidades reutilizables (Pressman, 2010).

En definitiva, la independencia funcional es la clave de tener un buen diseño de la calidad del programa. La independencia se evaluó según dos criterios cualitativos: la cohesión y el acoplamiento. La cohesión es un indicador de la fortaleza relativa funcional de un módulo. El acoplamiento lo es de la independencia relativa entre módulo (Pressman, 2010).

El módulo cohesión realiza una tarea, por lo que requiere interacción poco menos con otros componentes en otras partes del programa. En una palabra, un módulo cohesivo debería (idealmente) solo hacer una cosa. Aunque siempre se debe tratar de lograr mucha cohesión (por ejemplo, una sola tarea), a menudo necesaria y recomendada un componente de programa que realiza algunas funciones. Sin embargo, para un buen diseño hay que evitar componentes "esquizofrénicos" (módulos que realizan funciones no relacionadas) (Pressman, 2010).

El acoplamiento es una indicación de la interconexión entre unidades en una estructura de software del grado en el que se entra o se hace referencia a un módulo y de qué datos pasan a través de la interfaz. En el diseño de software, es posible que se requiera un emparejamiento mínimo. La conexión sencilla entre módulos da como resultado un programa que es más fácil de entender y menos propenso a propenso al "efecto de oleaje" [Ste74], se produce cuando se produce un error en un sitio web y se propaga por el sistema (Pressman, 2010).

1.1.2.8. Refinamiento

El refinamiento stepwise es una estrategia de diseño propuesta originalmente por Niklaus Wirth [Wir71]. El programa se crea ajustando los detalles del proceso sucesivamente. La jerarquía se desarrolla con la separación del enunciado macroscópico sobre la función (abstracción de procedimiento) de manera incremental hasta que se alcanzan las instrucciones lenguaje de programación. De hecho, el refinamiento es un proceso de elaboración, comienza con una declaración

de la función (o descripción de información), definida en un alto nivel de abstracción (Pressman, 2010).

Es decir, una frase que describe la función o información conceptual, pero no dice nada al respecto del funcionamiento interno de la estructura o función de la información interna. Entonces se trabaja en el enunciado original, brindando más y más detalles a medida que tiene lugar el refinamiento (elaboración) sucesivo (Pressman, 2010).

Abstracción y refinamiento son dos conceptos complementarios. El primero le permite seleccionar procedimientos y datos internos, pero elimina la necesidad de "extraños". El refinamiento ayuda a revelar detalles tales como avance de diseño. Ambos conceptos pueden crear un modelo completo del diseño según cómo evoluciona (Pressman, 2010).

1.1.2.9. Aspectos

A medida que avanza el análisis de necesidades, surgen una serie de "preocupaciones" que "incluye requisitos, casos de uso, características, estructuras de datos, calidad, servicios, Variables, Propiedad Intelectual Límites, Colaboración, Formularios y Contrato" [AOS07]. Idealmente, el modelo de requisitos debe estar estructurado de tal manera que permita aislar cada problema (solicitud) para una consideración independiente, sin embargo, en la práctica, algunas de estas preocupaciones afectan a todo el sistema y no son fáciles de resolver y dividirlos en compartimentos (Pressman, 2010).

Cuando comienza el diseño, los requisitos se refinan en la representación del diseño modular. Considere dos requisitos, A y B. A bloquea B "si se elige uno". El A interfiere con el B "si se ha elegido una descomposición [refinamiento] en la que B no puede satisfacerse sin tomar en cuenta a A" [Ros04] Por ejemplo, considere dos requisitos para la aplicación web CasaSeguraAsegurada.com. El requisito A se describe con el caso de uso de AVC-DVC. Un refinamiento del diseño se centrará en módulos que permitan a los usuarios registrados accedan a video desde cámaras colocadas en el espacio (Pressman, 2010).

El requisito B es de seguridad demostrar que los usuarios registrados deben estar autenticados antes de utilizar CasaSeguraAsegurada. com. Este requisito se aplica a todas las funciones disponibles para los usuarios registrados por CasaSegura. Cuando ocurre el refinamiento del diseño, A* representa el diseño para el requerimiento A y B* es otro diseño para el requerimiento B. Por lo tanto, A* y B* son representaciones de las preocupaciones, y B* interfiere con A* (Pressman, 2010).

El aspecto es una representación del problema de la intersección. Por lo tanto, representante del diseño, b* requiere que los primeros usuarios registrados se autenticuen usuario registrado debe ser

validado antes de que use CasaSeguraAsegurada.com es un aspecto de la aplicación web CasaSegura (Pressman, 2010).

Es importante especificar aspectos, de modo que el diseño pueda acomodarlos como una mejora conforme sucede el refinamiento y la división en módulos. En el escenario ideal, un aspecto se implementa como un módulo (componente) separado y no como fragmentos de software "dispersos" o "regados" entre múltiples componentes [Ban06]. Para lograr esto, la arquitectura de diseño debe soportar un mecanismo para definir aspectos: un módulo que permite la implementación de intereses en todos con las que interfiera (Pressman, 2010).

1.1.2.10. Rediseño

Una actividad de diseño importante recomendada para muchos métodos ágiles. Es el rediseño, técnica de reorganización para simplificar el diseño (o código) de un componente sin cambiar sus funciones o comportamiento. Fowler [Fow00] definen el rediseño en el modo de configuración: "Este es el proceso de modificar un sistema de software para que no cambie el comportamiento externo del código [diseño], pero sí se mejora su estructura interna" (Pressman, 2010).

Cuando se rediseña el programa, se revisa el diseño existente para la busca de redundancias, elementos de diseño no utilizados, algoritmos ineficientes o inútiles y estructuras de datos mal construido o inapropiadas, y cualquier otro defecto de diseño que pueda corregirse para lograr un mejor diseño. Por ejemplo, una primera iteración de diseño tal vez genere un componente con poca cohesión (realiza tres funciones que tienen poca relación entre sí). Después de un análisis cuidadoso, se decide rediseñar el componente en tres componentes separados, cada uno con mucha cohesión. El resultado será un software más fácil de integrar, de probar y de mantener (Pressman, 2010).

1.1.2.11. Conceptos de diseño orientados a objeto

El paradigma de la orientación a objeto (OO) se utiliza mucho en la ingeniería de software moderna. El apéndice 2 está pensado para aquellos lectores que no estén familiarizados con los conceptos de diseño OO, tales como clases y objetos, herencia, mensajes y polimorfismo, entre otros (Pressman, 2010).

1.1.2.12. Clases de diseño

El modelo requiere la identificación de un conjunto de elementos analíticos, cada uno describe algunos elementos del área del problema y se concentra en los aspectos visuales del problema para

los usuarios. El nivel de abstracción de una clase de análisis es relativamente alto. A medida que evoluciona el diseño, se define un conjunto de clase de diseño que refinan las clases de análisis, dando detalles del diseño que permitirá que las clases sean implementadas y generen una infraestructura de software para dar soporte a las soluciones empresariales. Puede desarrollar cinco tipos diferentes de diseño, cada uno representando una capa diferente de la ingeniería de diseño [Amb01]: (Pressman, 2010).

Tipos diferentes de clases de diseño cada una de las cuales representa una capa distinta de la arquitectura del diseño	
Clases de usuario de la interfaz.	Definen todas las abstracciones necesarias para la interacción humano-computadora (IHC). En muchos casos, la IHC ocurre dentro del contexto de una metáfora (por ejemplo, cuaderno de notas, formato de orden, máquina de fax, etc.) y las clases del diseño para la interfaz son representaciones visuales de los elementos de la metáfora.
Clases del dominio de negocios	Es frecuente que sean refinamientos de las clases de análisis definidas antes. Las clases identifican los atributos y servicios (métodos) que se requieren para implementar algunos elementos del dominio de negocios..
Clases de proceso.	Implantan abstracciones de negocios de bajo nivel que se requieren para administrar por completo las clases de dominio de negocios
Clases persistentes.	Representan almacenamientos de datos (por ejemplo, una base de datos) que persistirán más allá de la ejecución del software.
Clases de sistemas.	Implantan las funciones de administración y control del software que permiten que el sistema opere y se comunice dentro de su ambiente de computación y con el mundo exterior

Tabla 2: Tipos diferentes de clases de diseño

Fuente: (Pressman, 2010).

A medida que la arquitectura toma forma, el nivel de abstracción disminuye cuando cada clase de análisis se convierte en una representación del diseño. En otras palabras, las clases analíticas representan objetos de datos (y los servicios relacionados a los que se aplican) utilizando la terminología del dominio del negocio. Las clases de diseño introducen más detalles técnicos como guía de implementación. Arlow y Neustadt [Arl02] sugieren revisar cada clase de diseño para garantizar "bien formada". Definen **cuatro características de clases** de diseño bien formadas: (Pressman, 2010).

Completa y suficiente. La clase de diseño debe encapsular toda características y métodos que pueden esperarse razonablemente (basados en una interpretación comprensible del nombre de la clase) y está presente para la clase. Por ejemplo, la clase de escena se define para un programa de edición de video, solo estará completo si tiene todas las características y los métodos se integran adecuadamente con la creación de la escena de video. La suficiencia garantiza que la clase de diseñador contenga solo los métodos suficientes para lograr el objetivo ni más ni menos (Pressman, 2010).

Primitivismo. Los métodos asociados con la clase de planificación deben centrarse en el cumplimiento de servicio para la clase. Cuando el servicio se realiza de una manera, la clase no debe proporcionar otra forma de hacer lo mismo. Por ejemplo, la clase VideoClip para el software de edición de video puede tener atributos de punto-inicial y punto-final que se refieren a el punto inicial y final del clip (tenga en cuenta que el video original ya se cargó en el sistema, puede ser más largo que usar la abreviatura). Los métodos EstablecerPuntoInicial () y EstablecerPuntoFinal () proporciona la única forma de establecer el punto de inicio y terminación del video corto (Pressman, 2010).

Mucha cohesión. Una categoría de diseño consistente que contiene un pequeño conjunto de objetivos y responsabilidad; Para su implementación, se utilizan atributos y métodos de objetivos único. Por ejemplo, la clase VideoClip puede contener un conjunto de métodos para editar videos. La cohesión se conserva siempre que cada método se centre solo en los atributos relacionados en el corto (Pressman, 2010).

Poco acoplamiento. En el modelo de diseño lo que se requiere es que las clases de diseño cooperen entre sí. Sin embargo, la cooperación debe mantenerse al mínimo aceptable. Si un modelo de diseño está estrechamente acoplado (todas las clases de diseño cooperan con todos los demás sistemas), el sistema es difícil de configurar, probar y mantener a lo largo del tiempo. En general, las clases de diseño dentro de un subsistema deben tener sólo un conocimiento limitado de otras clases (Pressman, 2010).

Esta restricción se llama ley de Deméter [Lie03] y se sugiere que el método solo debe enviar notificaciones a métodos en clases adyacentes (Pressman, 2010).

1.2. EL MODELO DEL DISEÑO

El modelo del diseño puede verse en dos dimensiones distintas, como se ilustra en la ilustración 3. La dimensión del proceso indica la evolución del modelo del diseño conforme se ejecutan las tareas de éste como parte del proceso del software. La dimensión de la abstracción representa el nivel de detalle a medida que cada elemento del modelo de análisis

se transforma en un equivalente de diseño y luego se mejora en forma iterativa. En relación con la ilustración 3, la línea punteada indica la frontera entre los modelos de análisis y de diseño (Pressman, 2010).

En ciertos casos, es posible hacer una distinción clara entre ambos modelos. En otros, el modelo de análisis se mezcla poco a poco con el de diseño y la distinción es menos obvia (Pressman, 2010).

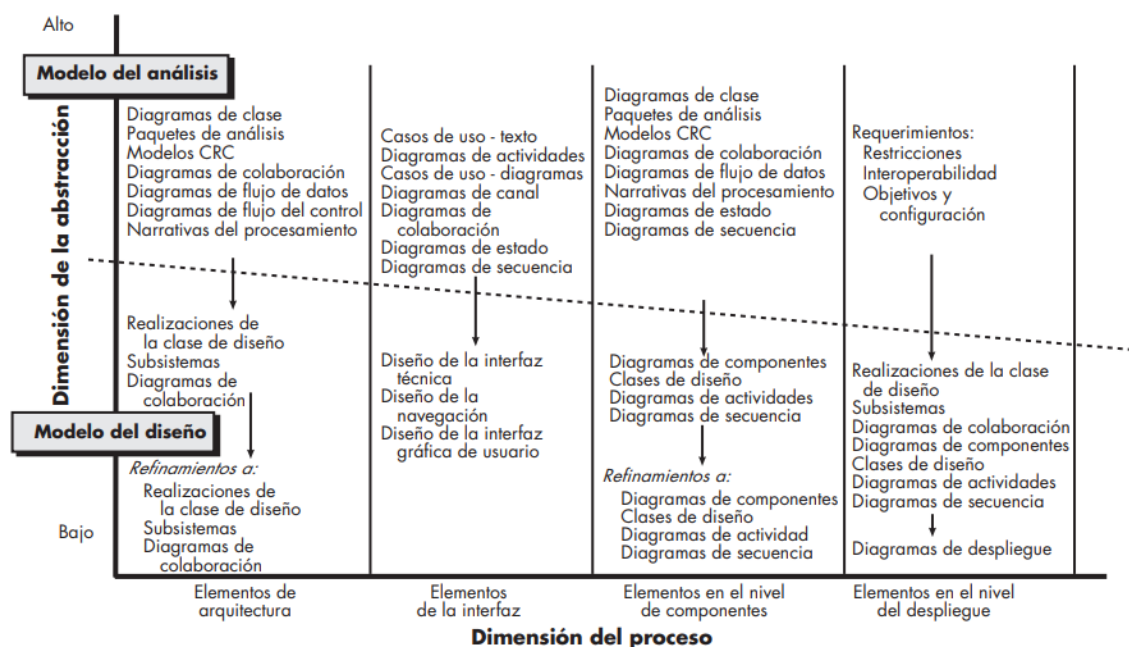


Ilustración 3: Dimensiones del modelo de diseño

Fuente: (Pressman, 2010).

Los elementos del modelo de diseño usan muchos de los diagramas UML7 que se utilizaron en el modelo del análisis. La diferencia es que estos diagramas se refinan y elaboran como parte del diseño; se dan más detalles específicos de la implantación y se hace énfasis en la estructura y en el estilo arquitectónico, en los componentes que residen dentro de la arquitectura y en las interfaces entre los componentes y el mundo exterior (Pressman, 2010).

No obstante, debe observarse que los elementos del modelo indicados a lo largo del eje horizontal no siempre se desarrollan en forma secuencial. En la mayoría de los casos, el diseño preliminar de la arquitectura establece la etapa y va seguido del diseño de la interfaz

y del diseño del nivel de los componentes, los cuales con frecuencia ocurren en paralelo. El modelo de despliegue por lo general se retrasa hasta que el diseño haya sido desarrollado por completo (Pressman, 2010).

Es posible aplicar patrones de diseño en cualquier punto de este proceso. Estos patrones permiten aplicar el conocimiento del diseño a problemas específicos del dominio que han sido encontrados y resueltos por otras personas (Pressman, 2010).

1.2.1. DISEÑO DE LA ARQUITECTURA:

En un importante libro sobre el tema, Shaw y Garlan [Sha96] presentaron el siguiente punto de vista sobre la ingeniería de software:

Dado que el software se dividió por primera vez en módulos, los sistemas de software tienen una arquitectura y los programadores son responsables de las interacciones entre módulos y propiedades del montaje. Históricamente, las arquitecturas han sido implícitas: fallas de implementación o sistemas obsoletos del pasado (Pressman, 2010).

Los buenos desarrolladores de software suelen adoptar uno o más estilos arquitectónicos como estrategia organizativa del sistema, pero lo usan de manera informal y de ninguna manera lo aclaran en el sistema resultante. Actualmente se ha diseñado, visualizado y diseñado claramente una arquitectura de software eficiente que se ha convertido en la disciplina dominante en la ingeniería de software (Pressman, 2010).

1.2.1.1. *Arquitectura del software.*

Al considerar la arquitectura de una construcción, llegan a la mente muchos atributos distintos. En el nivel más simple, se considera la forma general de la estructura del material, pero, de hecho, la arquitectura es mucho más que eso. Es la forma en que los diversos componentes de un edificio se combinan para formar un todo unificado. Aquí está cómo la construcción se adapta a su entorno y se fusiona con otros edificios de la zona. Este es el nivel en el que el edificio cumple bien su función y satisface las necesidades de sus propietarios. Es la sensación estética de la estructura —el efecto visual de la edificación— y el modo en el que se combinan texturas, colores y materiales para crear la fachada en el exterior y el “ambiente de vida” en el interior, son los pequeños detalles: diseño de la lámpara, tipo de suelo y color cortinas... la lista es casi interminable. Finalmente, es arte, pero

la arquitectura también es otra cosa. Estos es arte. Pero la arquitectura también es algo más. Son los “miles de decisiones, tanto grandes como pequeñas” [Tyt05]. Algunos se hacen en las primeras etapas del embarazo y tienen un efecto profundamente en todas las demás acciones. Otros quedan para después, con lo que se eliminan las limitaciones iniciales que pueden conducir a una mala implementación del estilo arquitectónico. Pero, ¿qué es la ingeniería de software? Bass, Clements y Kazman [Bas03] definen esto: La arquitectura de software de un programa o sistema de computadora es una arquitectura o estructuras del sistema, incluidos los componentes de software y sus propiedades visibles externamente, y relación entre ellos (Pressman, 2010).

La arquitectura no funciona con el software. Es una representación que permite 1) analizar la eficacia del diseño para cumplir requisitos específicos, 2) considerar alternativas arquitectónicas que se encuentra en una etapa en la que realizar cambios en el diseño aún es relativamente fácil y 3) Reducir los riesgos asociados con la construcción del programa (Pressman, 2010).

Esta definición enfatiza el papel de los "componentes de software" en cualquiera de ellos (Pressman, 2010).

Representación arquitectónica. Como parte del diseño arquitectónico, es uno de los componentes del software puede ser tan simple como una unidad de programa o una clase orientada a objetos, pero también puede extenderse a bases de datos y middleware que permiten la configuración de redes de clientes y servidores. Las Propiedades del componente son propiedades que son necesarias para entender cómo ciertos componentes interactúan con otros. A nivel de arquitectura, las características internas (como los detalles del algoritmo) no se especifican. La relación entre los componentes puede ser tan simple como un procedimiento de llamada de unidad a módulo o tan complejo como un protocolo de acceso en la base de datos (Pressman, 2010).

Algunos miembros de la comunidad de ingenieros de software [Kaz03] están marcando la diferencia entre las acciones involucradas en la obtención de una arquitectura de software (que el autor denominado "diseño arquitectónico") y los aplicados para obtener el diseño del software. Como dijo un revisor de esta edición: Hay una diferencia entre los términos arquitectura y diseño. Un diseño es una instancia de una arquitectura, de forma análoga a que un objeto sea una instancia de una clase. Por ejemplo, considere la arquitectura Cliente-servidor. Con esta arquitectura, es posible diseñar un sistema de software de muchas

maneras, un sistema de software basado en red, utilizando plataforma Java (Java EE) o Microsoft (.NET framework). Entonces, hay una estructura, pero sobre esta base se pueden crear muchos diseños. Así, no es correcto mezclar "Arquitectura y Diseño" (Pressman, 2010).

Aunque el autor entiende que el diseño de software es una instancia de una arquitectura de software particular, y los componentes y estructuras que forman parte de ella son la fuente de cualquier diseño resultante. El diseño comienza con la reflexión arquitectónica, el diseño de ingeniería de software considera dos niveles de la jerarquía de diseño (ver ilustración 1): Diseño de datos y diseño arquitectónico. En el contexto del análisis anterior, el diseño de datos nos permite representar la composición de los datos de una arquitectura con las definiciones usuales de sistemas y clases (incluyendo propiedades y operaciones) en un sistema orientado a objetos. El diseño arquitectónico se centra en expresar la estructura, las propiedades y las interacciones de los componentes del software (Pressman, 2010).

1.2.1.2. ¿Por qué es importante la arquitectura?

En un libro sobre ingeniería de software, Bass et al. [Bas03] identificó tres razones, las principales razones de la importancia de la ingeniería de software:

- Representaciones de arquitectura de software que permiten la comunicación entre todas las partes (participantes) interesadas en desarrollar un sistema basado en el ordenador.
- La arquitectura destaca las decisiones tempranas que tendrán un profundo impacto en todo trabajo posterior de ingeniería de software, e igual de importante, para el éxito final sistema como una entidad operativa.
- La arquitectura proporciona "un modelo relativamente pequeño e intelectualmente accesible de cómo se estructura y la forma en la que sus componentes trabajan juntos" [Bas03] (Pressman, 2010).

Modelos de diseño arquitectónico y modelos arquitectónicos en ellos es transferible. Esto significa que se le pueden aplicar tipos, estilos y estilos arquitectónicos (Pressman, 2010).

Diseña otros sistemas y representa un conjunto de elementos abstractos que permiten a los ingenieros de software describir la arquitectura de forma predecible (Pressman, 2010).

1.2.1.3. Géneros arquitectónicos.

Aunque los principios básicos de los diseños arquitectónicos se aplican a todos los tipos de arquitectura, a menudo el tipo de arquitectura que muestra el enfoque especificado para la estructura que debe ser construida. En el contexto del diseño arquitectónico, se relaciona el género relacionado con un tipo específico del dominio público del programa. En cada categoría hay diferentes subcategorías. Por ejemplo, en la categoría de construcción, existen los siguientes estilos generales: casas, condominios, edificios de apartamentos, edificios de oficinas, edificios de gran altura, industrias, almacenes, etc. Dentro de cada patrón general, hay patrones más específicos. Cada tipo tendrá una estructura que se puede describir usando una matriz de Patrones predecibles. En los documentos evolutivos para la ingeniería de software [boo08], Gray Boch sugirió los siguientes géneros arquitectónicas de sistemas basados en software: (Pressman, 2010).

Géneros arquitectónicos para sistemas basados en software	
Inteligencia artificial:	Sistemas que simulan o incrementan la cognición humana, su locomoción u otros procesos orgánicos.
Comerciales y no lucrativos:	Sistemas que son fundamentales para la operación de una empresa de negocios.
Comunicaciones:	Sistemas que proveen la infraestructura para transferir y manejar datos, para conectar usuarios de éstos o para presentar datos en la frontera de una infraestructura.
Contenido de autor:	Sistemas que se emplean para crear o manipular artefactos de texto o multimedia.
Dispositivos:	Sistemas que interactúan con el mundo físico a fin de brindar algún servicio puntual a un individuo.
Entretenimiento y deportes:	Sistemas que administran eventos públicos o que proveen una experiencia grupal de entretenimiento.
Financieros:	Sistemas que proporcionan la infraestructura para transferir y manejar dinero y otros títulos.
Juegos:	Sistemas que dan una experiencia de entretenimiento a individuos o grupos
Gobierno:	Sistemas que dan apoyo a la conducción y operaciones de una institución política local, estatal, federal, global o de otro tipo.
Industrial:	Sistemas que simulan o controlan procesos físicos.
Legal:	Sistemas que dan apoyo a la industria jurídica.
Médicos.	Sistemas que diagnostican, curan o contribuyen a la investigación médica.

Militares:	Sistemas de consulta, comunicaciones, comando, control e inteligencia (o C4I), así como de armas ofensivas y defensivas.
Sistemas operativos:	Sistemas que están inmediatamente instalados en el hardware para dar servicios de software básico.
Plataformas:	Sistemas que se encuentran en los sistemas operativos para brindar servicios avanzados.
Científicos:	Sistemas que se emplean para hacer investigación científica y aplicada.
Herramientas:	Sistemas que se utilizan para desarrollar otros sistemas.
Transporte:	Sistemas que controlan vehículos acuáticos, terrestres, aéreos o espaciales.
Utilidades:	Sistemas que interactúan con otro software para brindar algún servicio específico

Tabla 3: Géneros arquitectónicos para sistemas basados en software

Fuente: (Pressman, 2010).

Desde una perspectiva de diseño arquitectónico, cada tipo presenta un desafío único, Por ejemplo, considere la arquitectura de software de un sistema de juegos. Este tipo de sistema a veces llamadas *aplicaciones interactivas inmersivas*, requieren extensos algoritmos computacionales, gráficos de computadora avanzados, una fuente de datos de medios continuos, interacción en tiempo real a través de dispositivos de entradas convencionales y no convencionales, y otras preocupaciones especializadas (Pressman, 2010).

Alexandre François [FRA03] propone una arquitectura de software para Inmerpresencia¹ como se aplica al entorno del juego. Él describe la arquitectura de la siguiente manera:

ASI (Inmerpresence Software Architecture) es un nuevo modelo de arquitectura de software para diseñar, analizar e implementar aplicaciones que realizan procesamiento distribuido, asíncrono y paralelo de flujos de datos generales. El objetivo de ASI es proporcionar un marco universal para la implementación distribuida de algoritmos y su fácil integración en sistemas complejos [...] El modelo de procesamiento de datos extensible subyacente y el modelo de procesamiento híbrido (envío y reenvío de mensajes compartidos), asíncrono y paralelo permiten la manipulación natural y el uso eficiente de flujos de datos generales con indistinto uso de bibliotecas o código existentes original. La modularidad de tipos facilita el desarrollo, la prueba y la reutilización de código distribuido,

¹ Francois utiliza el término inmerpresencia para denotar aplicaciones interactivas de inmersión

así como el diseño e integración rápida del sistema, así como el mantenimiento y desarrollo (Pressman, 2010).

Un análisis detallado de ASI está más allá del alcance de este libro. Sin embargo, lo importante es darse cuenta de que el tipo de sistema de juego se puede abordar con un estilo arquitectónico diseñado específicamente para resolver problemas del sistema de juego (Pressman, 2010).

1.2.1.4. Estilos arquitectónicos.

Cuando un constructor usa el término "casa colonial" para describir una casa, la mayoría

Una sección de personas familiarizadas con la vivienda en los Estados Unidos creará una imagen pública y su capacidad de distribución. El constructor utilizó un estilo arquitectónico como un dispositivo descriptivo para distinguir una casa de otros estilos (por ejemplo, gablete, casa de campo, Cape Cod, etc.). Pero más importante es el estilo arquitectónico también es un modelo para la construcción. Necesita especificar más detalles, determinar su tamaño, terminar, agregar características personalizadas, especificar materiales de construcción, pero un estilo ("sala central colonial") dirige la mampostería en su obra. El software diseñado para sistemas informáticos también contiene uno de muchos tipos arquitectónicos. Cada tipo describe un tipo de sistema que consiste en 1) un conjunto de componentes (como una base de datos o una unidad computacional) que realizan una función requerida por el sistema, 2) un conjunto de conectores que permiten "comunicación, coordinación y colaboración" entre componentes, 3) restricciones que definen cómo se integran los componentes para formar un sistema, y 4) modelos semánticos que permiten a los diseñadores entienden las características generales del sistema analizando las características conocidas de sus partes componentes [Bas03] (Pressman, 2010).

El estilo arquitectónico es una transformación que se impone en el diseño de todo el sistema. El objetivo es crear una estructura para todos los componentes del sistema (Pressman, 2010).

La imposición de un estilo arquitectónico conducirá a cambios fundamentales en la arquitectura del software, incluida la reasignación de funciones de los componentes [Bos00] (Pressman, 2010).

Un patrón arquitectónico, como el estilo arquitectónico, impone la transformación conceptual de la arquitectura. Sin embargo, el estilo difiere del estilo en varios aspectos básicos: 1) el alcance del modelo es más limitado y se centra en un aspecto de la arquitectura en general, 2) un estilo arquitectónico impuesto por reglas, que describe cómo, donde el programa administrará algunos aspectos de su funcionalidad a nivel de infraestructura (por ejemplo, concurrencia) [Bos00], 3) patrones arquitectónicos tienden a abordar ciertos aspectos del comportamiento en un contexto arquitectónico, (por ejemplo, cómo manejarán las aplicaciones la sincronización o la interrupción en tiempo realmente). Los estilos se utilizan junto con los estilos arquitectónicos para formar la estructura general del sistema (Pressman, 2010).

1.2.1.5. Breve taxonomía de estilos de arquitectura

Aunque se han creado millones de sistemas informáticos en los últimos 60 años, estos pertenecen en gran medida a un número relativamente pequeño de estilos arquitectónicos:

Arquitectura del centro de datos. En el corazón de esta arquitectura hay un almacén de datos (como archivos o bases de datos) al que otros acceden con frecuencia otros componentes que actualizan, agregan, eliminan o modifican datos en un formato del almacenamiento. La ilustración 4 muestra un tipo común de centralización de datos. El software de cliente accede a un repositorio central. En algunos casos, es pasivo, ósea que el programa cliente accede a los datos independientemente de los cambios que sufran u otras acciones del software del cliente. Una variación de este enfoque transforma el repositorio en el "pizarrón" envía notificaciones al software del cliente cuando los datos cambian de interés del cliente (Pressman, 2010).

Las arquitecturas centradas en datos impulsan la integración [Bas03]. Esto significa que los componentes del programa pueden modificarse y agregarse, clientes, para arquitectura sin ningún problema con otros clientes (porque los componentes del cliente funcionan en forma independencia). Además, los datos se pueden transmitir entre los clientes utilizando el mecanismo pizararrón (los ingredientes de Pizarrón se utilizan para coordinar la transferencia de información entre clientes). Los componentes del cliente ejecutan procesos de forma independiente (Pressman, 2010).

Arquitecturas de flujo de datos. Esta estructura se aplica cuando los datos de entrada van convirtiéndolo en datos de salida mediante una serie de computacionales o manipuladores (Pressman, 2010).

Un patrón de tubo y filtro (ver ilustración 5) tiene un conjunto de componentes, se llaman filtros, conectados a tubos que pasan datos de un componente a otro. Cada filtro funciona independientemente de aquellos componentes que se localizan arriba o abajo del flujo; diseñado para predecir datos de entrada de un tipo específico y datos de salida de salida (al siguiente filtro) de una manera específica. Sin embargo, el filtro no requiere ningún conocimiento del trabajo realizado por los filtros vecinos (Pressman, 2010).

Si el flujo de datos se transforma en un flujo de conversión, se denomina lote secuencial. Esta arquitectura acepta un conjunto de datos y luego aplica una serie de componentes secuenciales (filtros) para transformarlos (Pressman, 2010).

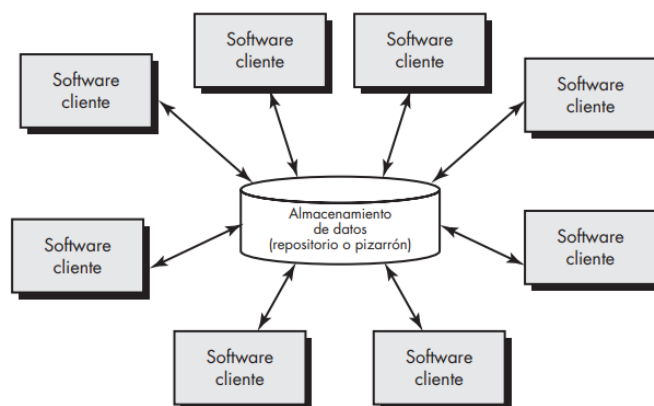


Ilustración 4: Arquitectura centrada en datos

Fuente: (Pressman, 2010).

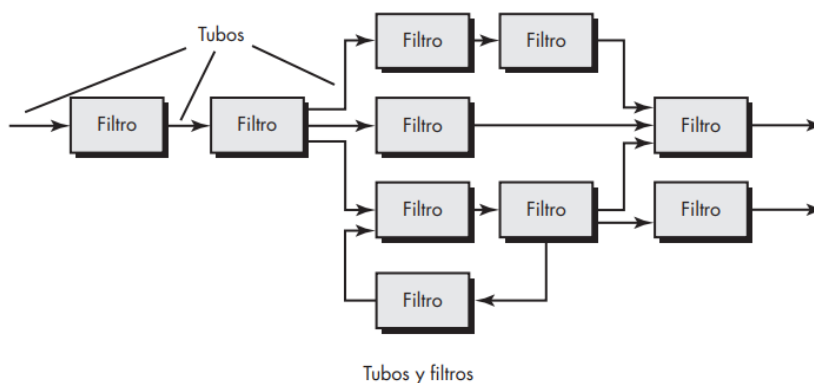


Ilustración 5: Arquitectura de flujo de datos

Fuente: (Pressman, 2010).

Arquitecturas de llamar y regresar. Este estilo arquitectónico permite una arquitectura de software que es relativamente fácil de modificar y escalar. En esta categoría hay varios subtipos [Bas03]:

- **Arquitecturas de programa principal/subprograma.** Esta es la estructura clásica del programa que divide el trabajo en una jerarquía de control, en la que hay un programa "principal", llama a algunos componentes del programa a su vez llama a otros. La ilustración 6 muestra una estructura de este tipo (Pressman, 2010).

- **Arquitecturas de llamada de procedimiento remoto.** Los componentes de la arquitectura del programa principal / subprograma están distribuida en la computadora múltiples en una red (Pressman, 2010).

Arquitecturas orientadas a objetos. Los componentes del sistema incluyen datos y operación que se deben aplicar para manipularlos. La comunicación y coordinación entre los componentes se implementan pasando mensajes (Pressman, 2010).

Arquitecturas en capas. En la ilustración 7 se ilustra la estructura básica de una arquitectura en capas. Se identifican varias categorías diferentes; Cada proceso realiza aproximaciones a las instrucciones de la máquina. En la capa exterior, los componentes manejan las operaciones de la interfaz de usuario. En la interna, los componentes realizan la interfaz con el sistema operativo. Las capas intermedias proporcionan servicios útiles funciones del programa de aplicación (Pressman, 2010).

Estos estilos arquitectónicos son solo un grupo de subconjunto pequeños disponibles. Cuando los requisitos técnicos muestran propiedades y restricciones del sistema que se desarrollará, el estilo arquitectónico o una mezcla de patrones que se ajusten mejor a esas características y restricciones. En muchos casos, más de un patrón es adecuado y posible diseñar y evaluar estilos arquitectónicos. Ejemplos en muchas aplicaciones de bases de datos combinan un estilo en capas (apropiado para la mayoría de sistemas) con una arquitectura centrada en datos (Pressman, 2010).

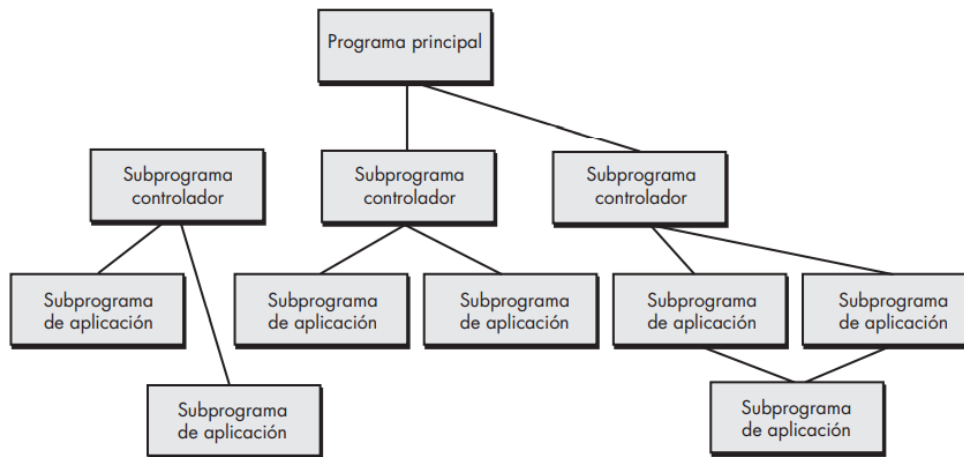


Ilustración 6: Arquitectura de programa principal/subprograma

Fuente: (Pressman, 2010).

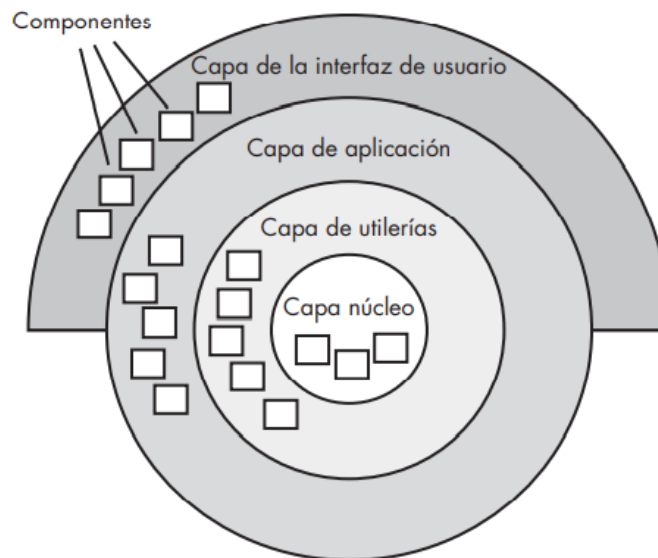


Ilustración 7: Arquitectura en capas

Fuente: (Pressman, 2010).

1.2.1.6. Patrones arquitectónicos.

Con el desarrollo del modelo de requisitos, se verá que el software debe resolver algunos problemas comunes que cubren toda la aplicación. por ejemplo, el modelo de requisitos para casi cualquier aplicación de comercio electrónico se enfrenta al siguiente problema: ¿Cómo ofrecemos una amplia gama de productos a un gran grupo de consumidores y les permitimos comprarlos en línea? El modelo de requerimiento también define un contexto que debemos

responder a esta pregunta. Por ejemplo, un negocio de comercio electrónico que venda equipo de golf de consumo operará en un contexto diferente que otro que venda equipo industrial de precio alto a corporaciones medianas y pequeñas (Pressman, 2010).

Además, existen diversas limitaciones y restricciones que afectan la manera de abordar este problema para resolverlo. Los modelos arquitectónicos resuelven un problema de aplicación específico en un contexto dado y sujeto a limitaciones y restricciones. El modelo propone una solución arquitectónica como base para el diseño arquitectónico. En la sección anterior, se dijo que la mayoría de las aplicaciones residen en un dominio o género, y uno o más estilos arquitectónicos que se ajusten a ellos. Por ejemplo, el patrón arquitectónico general de una aplicación puede ser el llamar y regresar o el que está orientado a objeto (Pressman, 2010).

Pero en este estilo encontrará una amplia gama de problemas generales que pueden resolverse mejor con modelos arquitectónicos específicos (Pressman, 2010).

1.2.1.7. *Diseño arquitectónico.*

Cuando comienza el diseño arquitectónico, se debe poner en él el software a desarrollar en contexto, es decir, el diseño debe identificar entidades externas (otros sistemas y dispositivos, personas, etc.) con las que el programa interactúa y la naturaleza de esa interacción. Esta información generalmente se obtiene del formulario de requisitos y todo lo encontrado durante su ingeniería. Una vez que el contexto es modelado y descrito en todas las interfaces de software externo, un conjunto específico de prototipos arquitectónicos. Un arquetipo es una abstracción (similar a una clase) que representa un componente conductual del sistema. Los arquetipos proporcionan una colección de elementos abstracto que debe diseñarse si se va a construir el sistema, pero los arquetipos por sí mismos no dan suficientes detalles para la implementación. Así, el diseñador define la estructura del sistema, identifica y refina los componentes de software que implementan cada arquetipo. Este proceso continúa repetidamente hasta que se obtiene una estructura arquitectónica completa (Pressman, 2010).

1.2.1.8. *Representación del sistema en contexto*

En el nivel de diseño arquitectónico, los ingenieros de software usan diagramas de contexto arquitectónico (DCA) para modelar cómo el software interactúa con entidades más allá de sus fronteras. Se muestra la estructura general del diagrama de contexto arquitectónico en la ilustración 8 (Pressman, 2010).

En relación con dicha figura, los sistemas interactúan con el sistema objetivo (aquel para el que se desarrollará el diseño arquitectónico) de la siguiente manera:

- **Sistemas superiores:** aquellos que utilizan al sistema objetivo como parte de algún esquema de procesamiento de alto nivel.
- **Sistemas subordinados:** los que son usados por el sistema objetivo y proveen datos o procesamiento que son necesarios para completar las funciones del sistema objetivo.
- **Sistemas entre iguales:** son los que interactúan sobre una base de igualdad (por ejemplo, la información se produce o consume por los iguales y por el sistema objetivo).
- **Actores:** entidades (personas, dispositivos, etc.) que interactúan con el sistema objetivo mediante la producción o consumo de información que es necesaria para el procesamiento de los requerimientos (Pressman, 2010).

Cada una de estas entidades externas se comunica con el sistema de destino a través de una interfaz (pequeño rectángulo gris). Para ilustrar el uso de DCA, considere la característica de seguridad del hogar del producto CasaSegura. El controlador general del producto CasaSegura y el sistema basado en Internet son por encima de la función de seguridad y por encima de la función se muestra en la ilustración 9 La función de monitoreo es un sistema entre iguales y, en versiones posteriores, el producto usa (usado por) la función de seguridad del hogar. Los paneles y consolas privados son los productores y consumidores de la información utilizada o producida por el Software de seguridad. Finalmente, los sensores son utilizados por el software de seguridad y se muestran como subordinados a éste (Pressman, 2010).

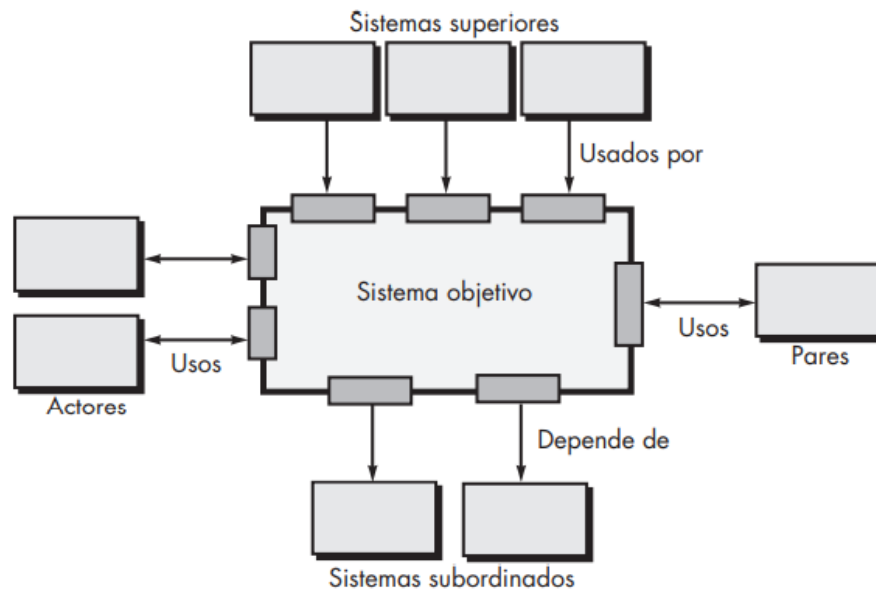


Ilustración 8: Diagrama de contexto arquitectónico 9.05

Fuente: (Pressman, 2010).

Como parte del diseño arquitectónico, tendrían que especificarse los detalles de cada interfaz mostrada en la ilustración 9. En esta etapa deben identificarse todos los datos que fluyen hacia dentro y fuera del sistema objetivo (Pressman, 2010).

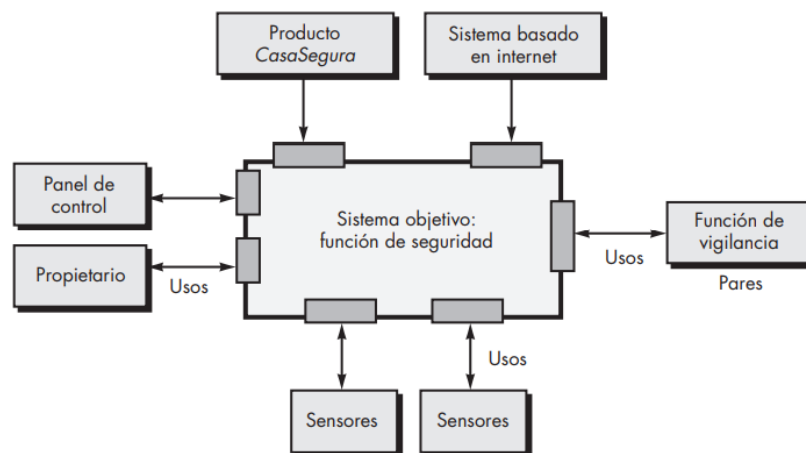


Ilustración 9: Diagrama de contexto arquitectónico para la función de seguridad de CasaSegura

Fuente : (Pressman, 2010).

1.2.1.9. Definición de arquetipos

Un arquetipo es una clase o patrón que representa a una abstracción básica que tiene una importancia fundamental para el diseño de la arquitectura del sistema de destino. En general, se requiere desde un conjunto de prototipos relativamente pequeños de arquetipos a fin de diseñar sistemas incluso algo complejos. La arquitectura del sistema de destino consta de estos arquetipos, que son elementos estáticos de la arquitectura, pero se implementan de diversas formas que varía según el comportamiento del sistema. En muchos casos, los arquetipos se obtienen estudiando categorías específicas de análisis como parte del modelado de requisitos. Al continuar con el análisis de la función de seguridad de CasaSegura se definirán los arquetipos siguientes: (Pressman, 2010).

- **Nodo.** Representa una colección cohesiva de elementos de entrada y salida de la función de seguridad del hogar. Por ejemplo, un nodo podría comprender 1) varios sensores y 2) varios indicadores de alarma (salida) (Pressman, 2010).
- **Detector.** Abstracción que incluye todos los equipos de detección que alimentan con información al sistema objetivo (Pressman, 2010).
- **Indicador.** Abstracción que representa todos los mecanismos (como la sirena de la alarma, luces, campana, etc.) que indican que está ocurriendo una condición de alarma (Pressman, 2010).
- **Controlador.** Abstracción que ilustra el mecanismo que permite armar o desarmar un nodo. Si los controladores residen en una red, tienen la capacidad de comunicarse entre sí (Pressman, 2010).

Cada uno de estos arquetipos se ilustra en la ilustración 10 usando notación UML. Recuerda que los arquetipos son la base de la arquitectura, pero son solo ideas abstractas que debe ser refinado a medida que avanza el diseño arquitectónico. Por ejemplo, el detector se refinará en una jerarquía de clases de sensores (Pressman, 2010).

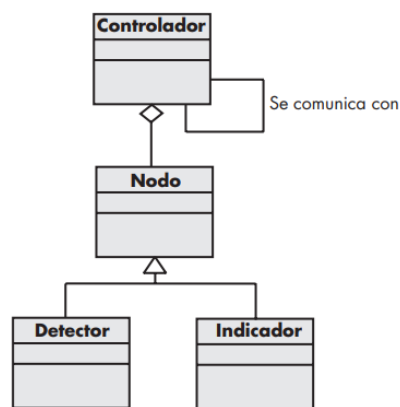


Ilustración 10: Relaciones de UML para los arquetipos de la función de seguridad de CasaSegura

Fuente: (Pressman, 2010).

1.2.2. DISEÑO EN EL NIVEL DE COMPONENTES:

El diseño del componente ocurre tan pronto como termina el diseño de la Arquitectura. En esta etapa, se creó la estructura general de los datos y los programas de Software. El objetivo es convertir un modelo de diseño en un programa de trabajo. Pero el nivel de abstracción del modelo de diseño actual es relativamente alto y el nivel del programa es bajo. La traducción es difícil y fácil de cometer pequeños errores que son difíciles de detectar y corregir más adelante en el proceso del software. En una famosa conferencia, Edsger Dijkstra, un destacado investigador que ha contribuido en gran medida a nuestra comprensión del diseño de software, dijo [Dij72]: (Pressman, 2010).

El software parece ser diferente de muchos otros productos en los que la regla es que a mejor calidad se da un mayor precio. Aquellos que desean un software en verdad confiable descubrirán que deben encontrar un medio para evitar de inicio la mayoría de los posibles errores; como resultado, el proceso de programación se hace más barato [...] los programadores eficaces no tienen que perder su tiempo depurando errores: no deben introducirlos al arrancar (Pressman, 2010).

Aunque estas palabras se dijeron hace muchos años, seguían siendo ciertas. Durante la traducción del modelo a código fuente, debe seguir los principios de diseño que no son solo traducción, sino también evite "ingresar errores desde el principio". Es posible representar el diseño en el nivel del componente con el uso de un lenguaje de programación (Pressman, 2010).

Básicamente, el software se crea utilizando el modelo de diseño arquitectónico como guía. Otro enfoque es representar el diseño al nivel de los componentes cuya representación mediana es (gráfica, tabular o textual) que sea fácil traducción al código fuente. Cualquiera que sea el mecanismo que se utilice para la representación a éste, las estructuras de datos, interfaces y algoritmos definidos deben apegarse a varios lineamientos de diseño bien establecidos que ayudan a evitar los errores conforme evoluciona el diseño del procedimiento (Pressman, 2010).

1.2.2.1. ¿Qué es un componente?

Un componente es un bloque de construcción de software de cómputo. Con más formalidad, la especificación OMG del Lenguaje de Modelado Unificado [OMG03a] define un componente como “una parte modular, desplegable y sustituible de un sistema, que incluye la implantación y expone un conjunto de interfaces”. los componentes forman la arquitectura del software y, en consecuencia, juegan un papel en el logro de los objetivos y de los requerimientos del sistema que se va a construir. Como los componentes se encuentran en la arquitectura del software, deben comunicarse y colaborar con otros componentes y con entidades (otros sistemas, dispositivos, personas, etc.) que existen fuera de las fronteras del software (Pressman, 2010).

El verdadero significado del término componente difiere en función del punto de vista del ingeniero de software que lo use. En las secciones que siguen, se estudian tres visiones importantes de lo que es un componente y cómo se emplea en el desarrollo de la modelación del diseño (Pressman, 2010).

1.2.2.2. Una visión orientada a objetos

En el contexto de la ingeniería de software orientada a objetos, el componente tiene un conjunto de clases colaborativas. Cada clase en un componente está completamente desarrollada para incluir todas las características y procesos asociados con su implementación. Como parte del proceso de desarrollo del diseño, todas las interfaces permiten a estas clases que se comunican y colaboran con otras clases de diseño. Para hacer esto, comience con el modelo de requisitos y cree clases de análisis (para componentes relevantes para el dominio del incidente) y clases de infraestructura (para los componentes

que brindan soporte para el dominio del incidente). Para ilustrar el proceso de desarrollo del diseño, considere el software desarrollado para una imprenta avanzada. El propósito general del software es cumplir con los requisitos del cliente en el mostrador, presupuesto para trabajos de impresión y luego muévelo a la instalación de producción automatizada. Durante la ingeniería de requerimientos, se obtuvo una clase de análisis denominada *ImprimirTrabajo*. En la parte superior la ilustración 11 muestra las características y actividades identificadas durante el análisis. En el diseño arquitectónico, *ImprimirTrabajo* se define como un componente de la arquitectura del software y se denota con el acrónimo UML que se muestra en la parte media derecha de la figura. Tenga en cuenta que *ImprimirTrabajo* tiene dos interfaces, *CalcularTrabajo*, que brinda la capacidad de obtener el costo del trabajo y *IniciarTrabajo*, y reenviar el trabajo a través de las instalaciones de producción. Se representan con símbolos de “paletas” que aparecen a la izquierda de la caja del componente (Pressman, 2010).

El diseño a nivel de componentes comienza desde este punto. Los detalles deben ser elaborados para el componente *ImprimirTrabajo* para que proporcione suficiente información para dirigir la ejecución. La clase de analizador se implementa inicialmente para representar todas las propiedades y operaciones necesarias para implementar la clase y el componente *ImprimirTrabajo*. En el interior de la parte inferior derecha de la ilustración 11, la clase de diseño elaborada *ImprimirTrabajo* contiene información más detallada sobre las propiedades, así como una descripción detallada de las operaciones que se ha requerido para implantar el componente. Interfaz *CalcularTrabajo* e *IniciarTrabajo* implica comunicación y colaboración con otros componentes (no mostrados). Por ejemplo, la operación *CalcularCostoPorPágina* () (que forma parte de la interfaz *CalcularTrabajo*) podría colaborar con un componente llamado *TabladeValuación* que contuviera información sobre los precios del trabajo. La operación *VerificarPrioridad* () (parte de la interfaz *IniciarTrabajo*) quizá colabore con un componente de nombre *FiladeTrabajos* para determinar los tipos y prioridades de trabajos que están en espera de su producción. Este procedimiento manual se aplica a cada elemento que se selecciona como parte del diseño de la arquitectura. Una vez concluida, se aplica más elaboración a cada atributo, operación e interfaz. Deben especificarse las estructuras de datos apropiadas para cada atributo. Además, se diseñan los detalles algorítmicos requeridos para implantar la lógica del procesamiento asociada con cada operación. Por último, se diseñan los mecanismos requeridos para implantar la interfaz.

Para el software orientado a objetos, esto incluye la descripción de todos los mensajes que se requieren para efectuar la comunicación dentro del sistema (Pressman, 2010).

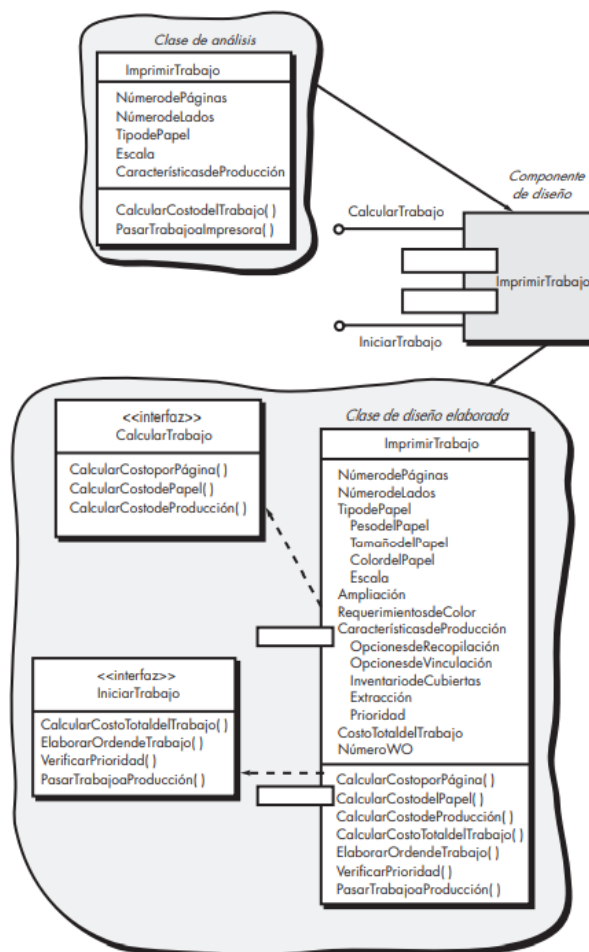


Ilustración 11: Elaboración de un componente de diseño

Fuente: (Pressman, 2010).

1.2.2.3. La visión tradicional

En el contexto de la ingeniería de software tradicional, un componente es un componente funcional de un programa que combina la lógica de procesamiento con las estructuras de datos internas necesarias para implementar la lógica de procesamiento y una interfaz que permite la invocación del componentes y transmisión de datos. En ingeniería de software se encuentra un componente convencional, también conocidos como módulos, tienen tres funciones importantes: (Pressman, 2010).

- 1) Como componente de control que coordina la invocación de todos los demás componentes del dominio del problema,
- 2) Como componente del dominio del problema que implanta una función completa o parcial que requiere el cliente y
- 3) Como componente de infraestructura que es responsable de las funciones que dan apoyo al procesamiento requerido en el dominio del problema (Pressman, 2010).

Al igual que los componentes orientados a objetos, los componentes de software tradicionales del modelo de análisis. Sin embargo, en este caso, el elemento de datos de la interfaz de usuario orientado al flujo del modelo de análisis es la base para su obtención. Cada transformación (burbuja) representado en los niveles inferiores del diagrama de flujo de datos mapeado en la jerarquía de módulos. Cerca de la parte superior de la jerarquía (la arquitectura de programa) son componentes de control (módulos) y hacia el fondo tienden a ser componentes del dominio del problema. Para lograr una modularidad efectiva, cuando se elaboran los componentes son conceptos de diseño detallados, como la independencia de funciones, que se aplican (Pressman, 2010).

Para ilustrar este proceso tradicional de desarrollo de diseño de componentes, considere una vez más, el software a desarrollar para una imprenta avanzada. Según los requisitos de modelado, se obtienen un conjunto de gráficos de flujo de datos. Suponga que se establece en la arquitectura que se puede ver en la ilustración 12. Cada rectángulo representa un componente del programa. Tenga en cuenta que el sombreado es equivalente en su función y operación a los definidos para la ImprimirTrabajo, sin embargo, en este caso, cada actividad se representa como un módulo aislado que se llama como se muestra en la figura. Para controlar el procesamiento se aplican otros módulos, por lo que son componentes de control. Cada módulo de la ilustración 12 se creó durante el proceso de diseño a nivel de componente. La Interfaz de unidad está bien definida. Esto significa que cada objeto de datos está representado por todo objeto de datos o control que pasa a través de la interfaz. Se define la estructura de datos utilizada en el interior del módulo. El algoritmo que permite que el modulo realice su función prevista está diseñado utilizando el enfoque de refinamiento por etapas. El comportamiento del módulo a veces se representa mediante un diagrama de estado. Para ilustrar este proceso, considere el módulo CalcularCostoporPágina.

El propósito de este módulo es calcular el costo de impresión por página de acuerdo a las especificaciones dadas por el cliente. Los datos necesarios para ejecutar esta función son: número de páginas en el documento, número total de documentos que se va a producir, impresión por uno o dos lados, requerimientos de color y requerimientos de tamaño. Estos datos se pasan a *CalcularCostoporPágina* a través de la interfaz del módulo. *CalcularCostoporPágina* utiliza estos datos para determinar el costo por página según el tamaño y complejidad del trabajo, en función de todos los datos proporcionados a la unidad a través de la interfaz. El costo por página es inversamente proporcional al tamaño de la página y el trabajo está a la altura de su complejidad. La ilustración 13 muestra el diseño a nivel de componentes utilizando la notación UML modificada. El módulo *CalcularCostoporPágina* accede a los datos llamando al módulo *ObtenerDatosdelTrabajo*, que permite pasar todos los datos relevantes al componente y la interfaz de la base de datos, *AccederBDdeCostos*, que permite a la unidad acceder a la base de datos que contiene todos los costos de impresión. Cuando el diseño progresa, se elabora el módulo *CalcularCostoporPágina* para que provea los detalles del algoritmo y de la interfaz. Los detalles del algoritmo se expresan utilizando un texto de símbolo falso que aparece en la imagen o con el esquema de actividad UML. Las interfaces están representadas como un grupo de organismos o conceptos de datos de entrada y salida. El desarrollo de diseño continua para que haya suficientes detalles para dirigir la construcción del componente (Pressman, 2010).

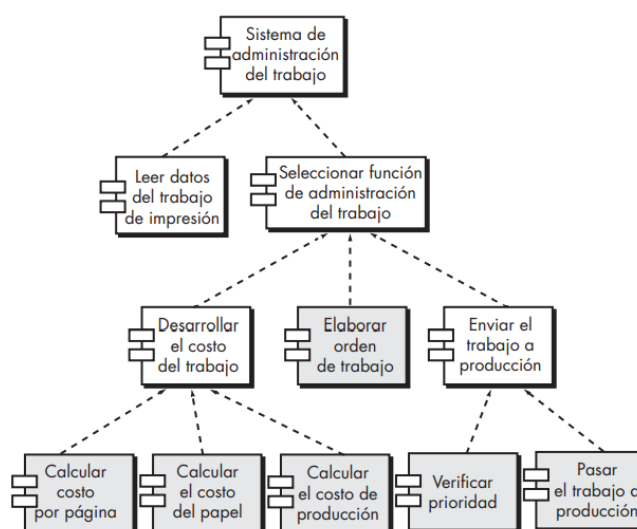


Ilustración 12: Gráfica de la estructura de un Sistema tradicional

Fuente: (Pressman, 2010).

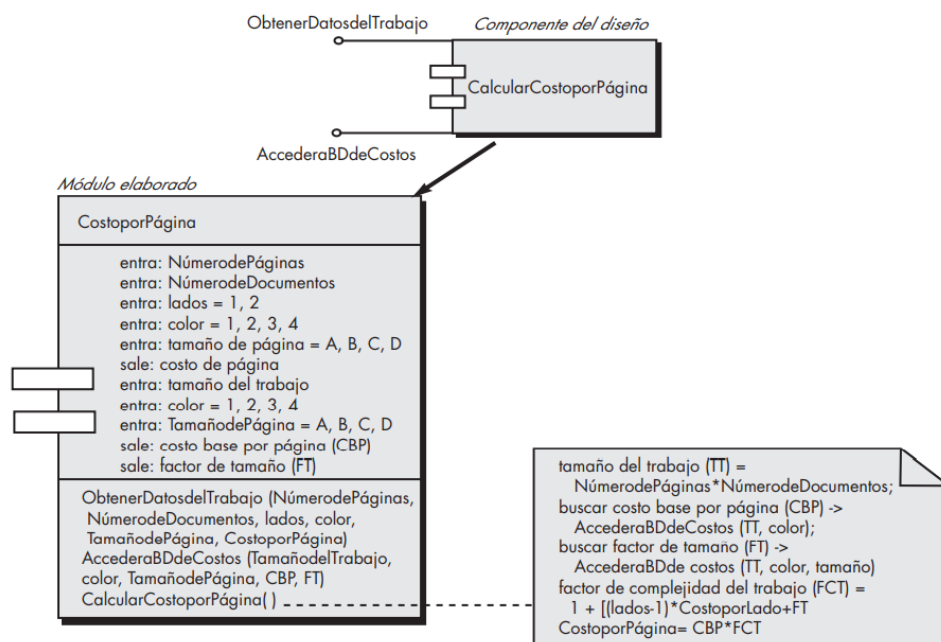


Ilustración 13: Diseño en el nivel de componentes para CalcularCostoporPágina

Fuente: (Pressman, 2010).

1.2.2.4. Visión relacionada con el proceso

La visión orientada a objetos y la tradicional a nivel de componentes, asumen que el componente fue diseñado desde la nada. Es decir, que se cree un nuevo componente en base a las especificaciones obtenidas de un modelo de requerimiento. Por supuesto, hay otro enfoque (Pressman, 2010).

En las últimas dos décadas, la comunidad de ingeniería de software se ha centrado en la necesidad de desarrollar sistemas utilizando componentes de software existentes o patrones de diseño. Esencialmente, a medida que avanza el trabajo de diseño, un catálogo de componentes diseñados o probados está disponible a nivel de código. Conforme se desarrolla la arquitectura de software, componentes o patrones de diseño son seleccionados y utilizados para construir la arquitectura. Estos componentes están contruidos teniendo en mente lo reutilizable, se dispone completamente para describir su interfaz, de las funciones que realizan y de la comunicación y colaboración que requieren (Pressman, 2010).

1.2.2.5. Elementos del diseño en el nivel de los componentes

El diseño a nivel de componente de software es equivalente a planos detallados (y especificaciones) para cada habitación de la casa. Estos diagramas muestran el cableado y Plomería en cada habitación, ubicación de cuadros eléctricos e interruptores, grifos, lavabos, duchas, bañeras, fregaderos, armarios y alacenas. También describen el tipo de piso que se utilizará, la decoración aplicada y todos los detalles relacionados con la habitación (Pressman, 2010).

El diseño de los componentes del programa describe completamente los detalles internos de cada componente. Para lograr esto, este diseño determina la estructura de datos de todos los objetos de datos los detalles y algoritmos locales que se realizan para todos los tratamientos en un componente, además de la interfaz que permite el acceso a todas las operaciones de los componentes (comportamientos) (Pressman, 2010).

En el contexto de las técnicas de software hacia el objeto, se muestra el componente en forma de gráfico UML, como se muestra en la ilustración 14. En él, aparece un componente llamado AdministracióndeSensor (parte de la función de seguridad de CasaSegura). La flecha punteada asocia el componente con una clase llamada sensor que se le asigna. El componente AdministracióndeSensor realiza funciones asociadas con los sensores de CasaSegura, incluso monitorearlos y configurarlos (Pressman, 2010).

Los detalles del diseño de componentes se modelan en un cierto nivel de abstracción diferente. El diagrama de actividad UML se utiliza para representar la lógica de procesamiento. El flujo de procedimiento detallado de un componente se representa como pseudocódigo (una representación similar a la representación de un lenguaje de programación) o en alguna otra forma diagramática (como diagramas de flujo o gráficos) (Pressman, 2010).

La arquitectura algorítmica sigue reglas establecidas de programación estructurada (por ejemplo, un conjunto de construcciones restringidas de procedimentales). Las estructuras de datos, seleccionadas de acuerdo con la naturaleza de los objetos de datos a procesar, por lo general, se construyen utilizando pseudocódigo del lenguaje de programación utilizado que se usará para la implementación (Pressman, 2010).

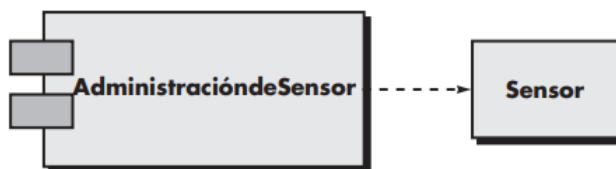


Ilustración 14: Diagrama de componente UML

Fuente: (Pressman, 2010).

1.2.2.6. Diseño de componentes basados en clase.

Como se mencionó, el diseño de nivel de componente basado en información se desarrolla como parte del modelo de requisitos expresado como parte del Modelo Arquitectónico, cuando se escoge un enfoque de ingeniería orientado al software, el diseño a nivel de componentes se enfoca en crear clases específicas del dominio problemático y ajustar las clases de infraestructura contenidas en el modelo de requerimientos. Una descripción detallada de los atributos, operaciones e interfaces que emplean estas clases son los detalles de diseño requeridos como precursor de la actividad de construcción (Pressman, 2010).

1.2.2.6.1. Principios básicos del diseño

Hay cuatro principios básicos que se aplican al diseño en términos de componentes que son ampliamente aceptado para aplicar ingeniería de software dirigida hacia el objeto (Pressman, 2010).

La principal motivación para la aplicación de estos principios es crear conceptos más factibles de cambiar, así como reducir la propagación de los efectos secundarios al hacer cambios. Estos principios se pueden utilizar como guía al desarrollar cada componente de software (Pressman, 2010).

Principios básicos de Diseño		
Principio	Abierto-Cerrado (PAC).	“Un módulo [componente] debe ser abierto para la extensión, pero cerrado para la modificación” [Mar00]. Este enunciado parece ser una contradicción, pero representa una de las características más importantes de un buen diseño en el nivel de componentes. Dicho en pocas palabras, debe especificarse el componente en forma tal que permita extenderlo (dentro del dominio funcional a que está dirigido)

	sin necesidad de hacerle modificaciones internas (en el nivel del código o de la lógica). Para lograr esto, se crean abstracciones que sirven como búfer entre la funcionalidad que sea probable extender y la clase de diseño en sí. Por ejemplo, suponga que la función de seguridad CasaSegura utiliza la clase Detector que debe revisar el estado de cada tipo de sensor de seguridad. Es probable que, conforme pase el tiempo, crezca el número y tipos de sensores de seguridad. Si la lógica de procesamiento interno se implementa como una secuencia de comandos si-entonces-en otro caso, cada uno dirigido a un tipo diferente de sensor, cuando se agregue uno nuevo se requerirá una lógica de procesamiento interno adicional (otro si-entonces-en otro caso). Esto sería una violación del PAC. En la ilustración 15 se ilustra una forma de seguir el PAC para la clase Detector. La interfaz sensor presenta una consistente visión de los componentes sensores para los detectores. Si se agregara un nuevo tipo de sensor, no se requeriría hacer ningún cambio para la clase Detector (componente). Se preservaría el PAC.
Principio de sustitución de Liskov (PSL).	“Las subclases deben ser sustituibles por sus clases de base” [Mar00]. Este principio de diseño, originalmente propuesto por Barbara Liskov [Lis88], sugiere que un componente que use una clase de base debe funcionar bien si una clase derivada de la clase base pasa al componente. El PSL demanda que cualquier clase derivada de una clase de base debe respetar cualquier contrato implícito entre la clase de base y los componentes que la usan. En el contexto de este análisis, un “contrato” es una precondition que debe ser verdadera antes de que el componente use una clase de base y una poscondición que debe ser verdadera después de ello. Cuando se crean clases derivadas hay que asegurarse de que respeten la precondition y la poscondición.
Principio de Inversión de la Dependencia (PID).	“Dependa de las abstracciones. No dependa de las concreciones” [Mar00]. Como se vio en el estudio del PAC, las abstracciones son el lugar en el que es posible ampliar un diseño sin muchas dificultades. Entre más dependa un componente de otros componentes concretos (y no de abstracciones tales como una interfaz), más difícil será ampliarlo.
Principio de segregación de la interfaz (PSI).	“Es mejor tener muchas interfaces específicas del cliente que una sola de propósito general” [Mar00]. Hay muchas instancias en las que múltiples componentes del cliente usan las operaciones que provee una clase servidor. El PSI sugiere que debe crearse una interfaz especializada que atienda a cada categoría principal de clientes. En la interfaz de ese cliente, sólo deben especificarse aquellas operaciones que sean relevantes para una categoría particular de clientes. Para las funciones de seguridad, PlanodelaCasa se utiliza sólo durante las actividades de configuración y emplea las operaciones SituarDispositivo(), MostrarDispositivo(), AgruparDispositivo() y QuitarDispositivo() para situar, mostrar, agrupar y quitar sensores del plano de la casa. La función de vigilancia de CasaSegura usa las cuatro operaciones mencionadas para la seguridad, pero también requiere operaciones especiales para administrar cámaras: MostrarFOV() y MostrarIdentificacióndeDispositivo(). Entonces, el PSI sugiere que los

	componentes cliente de las dos funciones de CasaSegura tienen interfaces especializadas definidas para ellas. La interfaz para la seguridad incluiría sólo las operaciones SituarDispositivo(), MostrarDispositivo(), AgruparDispositivo() y QuitarDispositivo(). La interfaz para vigilancia incorporaría esas mismas operaciones pero también MostrarFOV() y MostrarIdentificaciónDeDispositivo(). Aunque los principios de diseño en el nivel de componentes son una guía útil, los componentes no existen en el vacío. En muchos casos, los componentes o clases individuales están organizados en subsistemas o paquetes. Es razonable preguntar cómo debe ocurrir esta actividad de agrupamiento. ¿Exactamente cómo deben organizarse los componentes conforme avanza el diseño? Martin [Mar00] propone principios adicionales de agrupamiento que son aplicables al diseño en el nivel de componentes: Principio de equivalencia de la liberación de la reutilización (PER). “El gránulo de reutilización es el gránulo de liberación” [Mar00]. Cuando las clases o componentes se diseñan para ser reutilizables, existe un contrato implícito que se establece entre el desarrollador de la entidad reutilizable y las personas que la emplearán. El desarrollador se compromete a establecer un sistema que controle la liberación para que dé apoyo y mantenimiento a las versiones anteriores de la entidad mientras los usuarios se actualizan poco a poco con la versión más nueva. En lugar de abordar cada clase individual, es frecuente que sea mejor agrupar las que sean reutilizables en paquetes que puedan manejarse y controlar a medida que evolucionen las nuevas versiones. Principio de cierre común (PCC). “Las clases que cambian juntas pertenecen a lo mismo” [Mar00]. Las clases deben empacarse en forma cohesiva. Es decir, cuando las clases se agrupan como parte de un diseño, deben estar dirigidas a la misma área de funciones o comportamiento. Cuando deba cambiar alguna característica de dicha área, es probable que sólo aquellas clases que haya dentro del paquete requieran modificación. Esto lleva a un control de cambios y a un manejo de la liberación más eficaces. Principio de la reutilización común (PRC). “Las clases que no se reutilizan juntas no deben agruparse juntas” [Mar00]. Cuando cambia una o más clases dentro de un paquete, cambia el número de liberación del paquete. Entonces, todas las demás clases o paquetes que permanecen en el paquete que cambió deben actualizarse con la liberación más reciente y someterse a pruebas a fin de garantizar que la nueva versión opera sin problemas. Si las clases no se agrupan de manera cohesiva, es posible que se cambie una clase sin relación junto con las demás que hay dentro del paquete. Esto generará integración y pruebas innecesarias. Por esta razón, sólo las clases que se reutilicen juntas deben incluirse dentro de un paquete.
--	--

Tabla 4: Principios básicos de Diseño

Fuente: (Pressman, 2010).

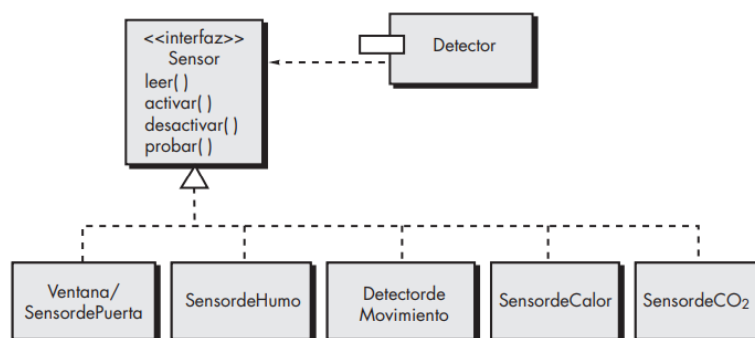


Ilustración 15: Seguimiento del PAC

Fuente: (Pressman, 2010).

1.2.2.7. Diseño en el nivel de componentes para WEBAPPS.

Por lo general, cuando se trata de sistemas y aplicaciones web, la línea entre el contenido y las funciones no están claros. Por lo tanto, tiene sentido hacer la pregunta: ¿qué es un componente de WEBAPPS? (Pressman, 2010).

Para los propósitos de esta clase, un componente de WEBAPPS es:

1) una función bien unida bien definida que manipula contenido o realizar operaciones aritméticas o de procesamiento de datos de usuario (Pressman, 2010).

2) un paquete cohesivo de contenido y funcionalidad que proporciona al usuario final alguna capacidad requerida. Por lo tanto, el diseño de aplicaciones web suele estar a nivel de componentes de WEBAPPS que Incorpora contenido y elementos de diseño de funciones (Pressman, 2010).

1.2.2.8. Diseño del contenido en el nivel de componente

El diseño de contenido a nivel de componente se enfoca en objetos de contenido y en cómo se empaqueta para su presentación al usuario final de la WEBAPPS. por ejemplo, considere la capacidad de vigilancia con video dentro de CasaSeguraAsegurada.com. Entre muchas otras capacidades, los usuarios eligen y controlan todas las cámaras que representan en el plano de la casa y solicita imágenes inmediatas de todos las cámaras con video de uno de ellos. Además, los usuarios tienen la capacidad de abrir el ángulo o de hacer

acercamientos con una cámara por medio de los íconos apropiados de control. Para la capacidad de vigilancia con video, se pueden identificar muchos componentes potenciales de contenido: (Pressman, 2010).

1) Objetos de contenido que representan la distribución del espacio (el plano de la casa) con íconos adicionales que representan la ubicación de sensores y cámaras de video,

2) un conjunto de imágenes de secuencias de video (cada una un objeto de datos separado) y

3) Una ventana de video para una cámara específica. Se da un nombre a cada uno de estos componentes por separado y tratados como un solo paquete (Pressman, 2010).

Considere el plano de la casa con cuatro cámaras colocadas estratégicamente en una casa. A pedido del usuario, se captura un video de cada cámara y se identifica como un objeto de contenido generado dinámicamente, TomadeVideoN, donde N selecciona las cámaras 1-4. Un elemento de origen, llamado ImágenesInstantáneas, combina cuatro objetos de contenido de TomadeVideoN y muéstrellos en su página de vigilancia con video. La apariencia del diseño de contenidos a nivel de componentes debe adaptarse a las características de la aplicación web que se está desarrollando. En muchos casos, los objetos de contenido no deben organizarse como componentes y pueden manipularse individualmente. Sin embargo, con el aumento del tamaño y la complejidad (para una aplicación web, los objetos de contenido y sus interrelaciones), es necesario organizar el contenido para que sea fácil hacer referencia y manipular el diseño. Además, si el contenido es muy dinámico (p. ej.: desde el sitio de subastas en línea), es importante establecer un modelo estructural claro que incluya elementos de contenido (Pressman, 2010).

1.2.2.9. *Diseño de las funciones en el nivel de componentes*

Las aplicaciones web modernas ofrecen capacidades de procesamiento cada vez más sofisticadas: 1) producen un procesamiento de localización de la creación de contenido, y capacidad de navegación Dinámica 2) Proporcionar capacidades informáticas o de procesamiento de datos que resultan adecuados con el negocio de aplicaciones web; 3) Asesoramiento y acceso avanzado a base de datos y 4) Crear una interfaz de datos con sistemas empresariales el exterior. Para lograr las capacidades anteriores (y más), los componentes de las aplicaciones web tienen la misma apariencia que los componentes de

software regulares. La funcionalidad de las aplicaciones web se proporciona como una serie de componentes que se desarrollan en conjunto con la estructura de la información que asegura su consistencia. Es necesario, que comience considerando la estructura inicial y el modelo de requisitos para la información, y luego estudie cómo los trabajos afectan la interacción del usuario con la aplicación, la información facilitada y las tareas realizadas por el usuario. Al diseñar la arquitectura, el contenido y la funcionalidad de la aplicación web se integran para crear una arquitectura funcional, que es una representación del área funcional de una aplicación web y describe los principales componentes funcionales de una aplicación web y cómo interactúan una con otra. Por ejemplo, las funciones de abrir el ángulo y hacer acercamientos de la capacidad de vigilancia con video para CasaSeguraAsegurada.com se implementan como las operaciones `recorrer()` y `zoom()`, que forman parte de la clase Cámara. En cualquier caso, la funcionalidad implícita para `recorrer()` y `zoom()` deben implantarse como módulos dentro de CasaSeguraAsegurada.com (Pressman, 2010).

1.2.2.10. Diseño de componentes tradicionales.

Los cimientos del diseño a nivel de componentes para el software tradicional se establecieron a principios de la década de 1960 y se formalizaron a través del trabajo de Edsger Dijkstra et al. ([Boh66], [Dij65] y [Dij76b]). Al final de esta etapa, sugieren usar un conjunto de construcciones lógicas restringidas sobre las cuales se puede construir cualquier programa. Las estructuras enfatizan la “mantenimiento del dominio funcional”. Es decir: cada construcción tiene una estructura lógica predecible, que se ingresa al principio y al salir al final, facilitando al lector seguir el progreso (Pressman, 2010).

Las construcciones son secuencia, condición y repetición. La secuencia implementa pasos de procesamiento requeridas en la especificación de cualquier algoritmo. La condición proporciona un medio para seleccionar un controlador basado en un evento booleano y La repetición permite hacer bucles, la ejecución de lazos. Estas tres construcciones son esenciales para la Programación estructurada, una importante técnica de diseño a nivel de componentes. Se han propuesto estructuras para limitar el diseño de software orientado a procedimientos a un pequeño número de estructuras lógicas predecibles. La Medición de la complejidad indica que el uso de trabajos estructurados alcanza la complejidad del programa y, por lo tanto, mejora la lectura y la facilidad de prueba y el apoyo a la producción. Usando un número limitado de obras lógicas que contribuye al proceso de comprensión del ser

humano que los psicólogos denominan lotificación. Para entender este proceso, considere al lector mientras lee esta página. Los niños no necesitan leer cada letra, pero deben familiarizarse con los patrones o combinaciones de letras que forman palabras u oraciones. Las estructuras estructuradas son conjuntos lógicos que permiten a los lectores aprender sobre los elementos de procedimiento de una unidad, en lugar de leer esquemas o líneas de código línea por línea. La comprensión mejora cuando se encuentran patrones lógicos reconocibles. Con solo tres estructuras organizadas, es posible diseñar e implementar cualquier programa, independientemente del campo de aplicación o complejidad técnica. Sin embargo, cabe señalar que su uso doctrinal a menudo provoca dificultades en la práctica (Pressman, 2010).

1.2.2.11. Desarrollo basado en componentes.

En el contexto de la ingeniería de software, la reutilización es una idea vieja y nueva. Los programadores reutilizaron ideas, abstracciones y operaciones desde los primeros días de las computadoras, pero su enfoque original es excepcional. Los sistemas informáticos complejos de alta calidad de hoy en día deben desarrollarse en plazos muy breves que requieren un enfoque más sistemático para la reutilización (Pressman, 2010).

La ingeniería de software basada en componentes de software (ISBC), es un proceso que enfatiza el diseño y construcción de sistemas informáticos utilizando "componentes" reutilizables. Clements [Cle95] describe ISBC de la siguiente manera: (Pressman, 2010).

La ISBC adopta la filosofía "compra, no hagas" propuesta por Fred Brooks y otros. Del mismo modo que las subrutinas de los primeros tiempos liberaron al programador de pensar en los detalles, la ISBC traslada el énfasis de la programación a la combinación de sistemas de software. La implantación ha dado paso a la integración como el aspecto importante (Pressman, 2010).

Pero surgen algunas preguntas. ¿Es posible construir sistemas complejos con el ensamble de componentes reutilizables de software procedentes de un catálogo? ¿Se logra esto en forma eficaz en cuanto a costo y tiempo? ¿Pueden establecerse incentivos apropiados que estimulen a los ingenieros de software a reutilizar, más que a reinventar? ¿La dirección está dispuesta a incurrir en los gastos adicionales asociados a la creación de componentes de software reutilizables? ¿Es posible crear la biblioteca de componentes necesarios para

conseguir la reutilización en forma tal que sea accesible a quienes los necesitan? ¿Quiénes necesitan los componentes que ya existen pueden encontrarlos? (Pressman, 2010).

La respuesta a estas preguntas es cada vez más un "sí". Hay ciertos aspectos a considerar para el éxito de ISBC en una institución de ingeniería de software (Pressman, 2010).

1.2.2.12. Ingeniería del dominio

El objetivo de la ingeniería del dominio es identificar, construir, catalogar y diseminar un conjunto de componentes de software que sean aplicables al software existente y al del futuro en un dominio particular de aplicaciones. La meta general es establecer mecanismos que permitan que los ingenieros de software compartan dichos componentes —los reutilicen— cuando trabajen en sistemas nuevos o ya existentes. La ingeniería del dominio incluye tres actividades principales: análisis, construcción y diseminación (Pressman, 2010).

El enfoque general del análisis del dominio se caracteriza con frecuencia dentro del contexto de la ingeniería de software orientada a objetos. Los pasos de este proceso se definen como sigue: (Pressman, 2010).

1. Definir el dominio que se va a investigar.
2. Clasificar los aspectos extraídos del dominio.
3. Reunir una muestra representativa de aplicaciones en el dominio.
4. Analizar cada aplicación en la muestra y definir clases de análisis.
5. Desarrollar un modelo de los requerimientos para las clases.

Es importante observar que el análisis del dominio es aplicable a cualquier paradigma de la ingeniería de software y se aplica tanto al desarrollo convencional como al orientado a objetos (Pressman, 2010).

1.2.3. DISEÑO DE LA INTERFAZ DE USUARIO:

El diseño de la interfaz de usuario crea un medio eficaz de comunicación entre los seres humanos y la computadora. Siguiendo un conjunto de principios de diseño de la interfaz, el diseño identifica los objetos y acciones de ésta y luego crea una plantilla de pantalla que constituye la base del prototipo de la interfaz de usuario (Pressman, 2010).

Donald Norman [Nor88] afirmaba que ya era tiempo de cambiar de actitud:

Para producir tecnología que se adapte a los seres humanos, es necesario estudiar a éstos. Pero en la actualidad tendemos a estudiar sólo a la primera. El resultado es que se exige a las personas que se adapten a la tecnología. Es tiempo de que esta tendencia se revierta, es el momento de que la tecnología se adapte a las personas (Pressman, 2010).

A medida que los tecnólogos estudiaban la interacción humana, surgieron dos aspectos dominantes. El primero fue que se identificaron reglas doradas (que se estudian en la sección 11.1). Éstas se aplican a toda interacción humana con productos de la tecnología. El segundo fue que se definió un conjunto de mecanismos de interacción que permitieron a los diseñadores de software construir sistemas que implantaban en forma correcta las reglas doradas. Esos mecanismos de interacción, llamados de manera colectiva interfaz gráfica del usuario (GUI), eliminaron algunos de los problemas más notables asociados con las interfaces humanas. Pero aun en un “mundo Windows”, todos hemos encontrado interfaces de usuario que son difíciles de aprender y usar, que son confusas y van contra la intuición, que no perdonan y, en muchos casos, resultan frustrantes por completo. No obstante, alguien dedicó tiempo y energía a la elaboración de estas interfaces, y no es probable que su constructor haya creado dichos problemas con toda intención (Pressman, 2010).

1.2.3.1. Elementos de diseño de la interfaz

El diseño de la interfaz del programa es similar al conjunto de características (y especificaciones) de detalles de puertas, ventanas y muebles para el hogar. Estos dibujos muestran el tamaño y la forma de puertas y ventanas, cómo funcionan y cómo son las instalaciones de servicios (agua, luz, gas, teléfono, etc.) se encuentran sobre la vivienda y se distribuyen entre las habitaciones especificadas en el esquema. Dicen dónde está el timbre, entonces se utilizará el intercomunicador para anunciar la presencia de invitados y cómo se instalará el sistema de seguridad. Básicamente, el esquema (y las especificaciones) de la puerta, las ventanas y las comodidades exteriores nos dicen cómo fluyen las cosas y la información en el interior y fuera de la casa y dentro de las habitaciones en el esquema (Pressman, 2010).

Los elementos de diseño de la interfaz del software permiten que la información fluya hacia dentro y fuera del sistema, y cómo están comunicados los componentes que son parte de la arquitectura. Hay tres elementos importantes en el diseño de la interfaz: 1) la interfaz

de usuario (IU), 2) interfaces externas relacionadas con otros sistemas, dispositivos, redes y productores o consumidores de información, y 3) interfaces internas relacionadas con otros componentes de diseño. Estos elementos de diseño de interfaz permiten que el programa se comunique con el exterior y permitir la comunicación interna y la colaboración entre los componentes que componen la arquitectura del programa (Pressman, 2010).

El diseño de interfaz de usuario (cada vez más conocido como diseño de usabilidad) es un diseño con la capacidad de utilizar una variedad de elementos estéticos (como diseño, colores, gráficos y mecánica de interacciones, etc.), elementos ergonómicos (como el diseño y la ubicación de la información, metáforas, navegación de la interfaz de usuario, etc.), elementos técnicos (como patrones de comunicación, interfaz de usuario y moldes reutilizables). En general, la interfaz de usuario es un subsistema en la arquitectura general de la aplicación. El diseño de la interfaz externa requiere información final sobre la entidad que se envíe la información o desde la que se recibe. En todos los casos, esta información debe recopilarse como ingeniería de ingeniería y se logra una vez que comienza el diseño de la interfaz (Pressman, 2010).

El diseño de las interfaces externas debe incluir el examen en busca de errores y (si es necesario) las medidas de seguridad apropiadas (Pressman, 2010).

El diseño de las interfaces internas está estrechamente relacionado con el diseño de los componentes. Los logros del diseño de categorías analíticas representan a todas las operaciones y esquemas de mensajería que se requiere para permitir la comunicación y la cooperación entre las actividades de las diferentes clases. Cada mensaje debe estar diseñado para que contenga información necesaria para el transporte y requisitos específicos para operar el proceso requerido. Si para el diseño, se elige enfoque clásico el proceso de entrada, la interfaz de cada componente del software está diseñada sobre la base del flujo de datos y ofertas de trabajo descritas en una cuenta de procedimiento (Pressman, 2010).

En algunos casos, la interfaz está diseñada más como una clase. En el UML una interfaz se define como [OMG03a]: "Es específica de los procesos [genéricos] que son visibles externamente para una clase, componente u otro clasificador (incluidos los subsistemas), sin especificar su estructura interna". En resumen, la interfaz es una colección de operaciones que describen y brindan acceso a una parte del comportamiento de una clase. Por ejemplo, la función de seguridad CasaSegura utiliza el panel de permisos que el propietario controle determinados aspectos de la función de seguridad. En la versión mejorada del sistema, las

funciones del panel de control se pueden realizar a través de un PDA inalámbrico o un teléfono móvil (Pressman, 2010).

La clase **PaneldeControl** (ilustración 16) proporciona el comportamiento asociado con un teclado, por lo que debe implementar las operaciones **LeerTeclazo ()** y **DecodificarTecla ()**. Si estas operaciones se van a dar a otras clases (en este caso, **PDAInalámbrico** y **TeléfonoMóvil**), es útil definir una interfaz como la de la figura. La interfaz, llamada **Teclado**, se ilustra como un estereotipo <> o como un círculo pequeño con leyenda y conectado a la clase con una línea. La interfaz se define sin atributos y con el conjunto de operaciones que sean necesarias para lograr el comportamiento de un teclado (Pressman, 2010).

La línea punteada con el triángulo abierto al final (en la ilustración 16) indica que la clase **PaneldeControl** proporciona acciones de teclado como parte de su comportamiento (Pressman, 2010).

En UML, esto se caracteriza como una realización. Es decir, parte del comportamiento de **PaneldeControl** se implementará con la realización de las operaciones de **Teclado**. Éstas se darán a otras clases que accedan a la interfaz (Pressman, 2010).

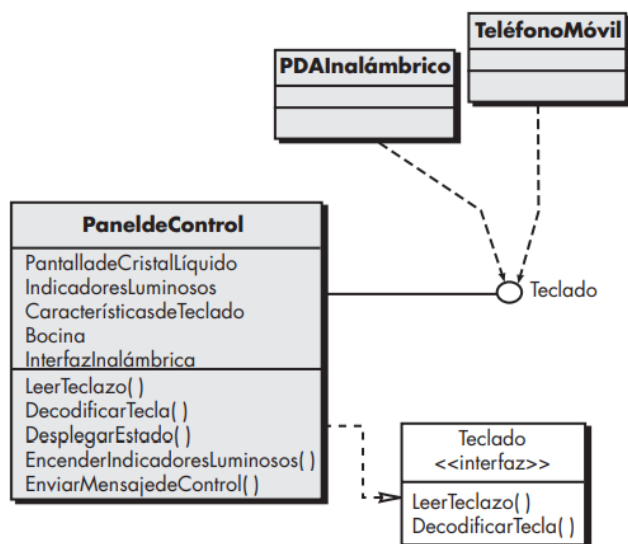


Ilustración 16: Representación de la interfaz para Panel de Control

Fuente. (Pressman, 2010).

1.2.3.2. Reglas.

En su libro sobre el diseño de la interfaz, Theo Mandel [Man97] acuñó tres reglas doradas:

1. Dejar el control al usuario.
2. Reducir la carga de memoria del usuario.
3. Hacer que la interfaz sea consistente.

En realidad, estas reglas doradas constituyen la base de un conjunto de principios de diseño de la interfaz de usuario que guían este aspecto tan importante del diseño del software (Pressman, 2010).

1.2.3.3. Dejar el control al usuario

Durante una sesión para recopilar requisitos para un gran sistema de información nuevo, se le preguntó a un usuario importante sobre los atributos de una GUI basada en ventanas (Pressman, 2010).

Él respondió solemnemente: "Lo que realmente quiero es un sistema que lea mis pensamientos. Saber lo que quiero hacer antes de que lo necesite y hacérmelo más fácil que quiero. Eso es todo." Mi primera reacción fue asentir y sonreír, pero me detuve un momento. No hay absolutamente ningún desperdicio de las solicitudes de los usuarios. Quería un sistema que satisfaga sus necesidades y ayúdelo a hacer las cosas. Quería controlar la computadora, no porque él la estuviera controlando. La mayoría de las restricciones y limitaciones establecidas por el diseñador tienen como objetivo simplificar la forma de interacción. ¿Pero para quién? Como diseñador, puede verse tentado a introducir limitaciones y restricciones que simplifiquen la implementación de la interfaz. El resultado puede ser una interfaz fácil de construir, pero que sea frustrante de usar. Mandel [Man97] identifica algunos principios de diseño que permite a los usuarios tener el control: (Pressman, 2010).

Definir modos de interacción de manera que no se obligue al usuario a realizar acciones innecesarias o no deseadas: El modo interactivo es el estado actual de la interfaz. Por ejemplo, si en la lista de procesadores de texto, se determina revisar los elementos de ortografía flexibles al modo de dictado. No hay razón para obligar al usuario a Perma Necer en este modo si desea crear una versión pequeña del texto. El usuario debe ser capaz de entrar y salir con poco o ningún esfuerzo (Pressman, 2010).

Dar una interacción flexible: Porque diferentes usuarios tienen diferentes preferencias para interactuar, se deben tomar decisiones. Por ejemplo, el programa debe permitir con el que el usuario interactúa con los comandos ingresados usando el teclado y el mouse, Lápiz digitalizador, pantalla táctil o mecanismo de reconocimiento de voz. Pero no se puede acceder a todas las acciones a través del mecanismo de interacción. Por ejemplo, considere la dificultad de usar comandos de teclado (o entrada de voz) para crear un dibujo complejo (Pressman, 2010).

Permitir que la interacción del usuario sea interrumpible y también reversible: El usuario debe poder detener el tema de su trabajo (incluso si incluye un hilo de acciones) para hacer otra cosa (sin perder el trabajo realizado hasta el momento). Además, debe poder "deshacer" cualquier acción (Pressman, 2010).

Facilitar la interacción a medida que aumenta la habilidad y permitir que aquella se personalice: Los usuarios a menudo hacen la misma secuencia de interacciones de forma repetida. Sería útil diseñar un mecanismo "macro" que permita al usuario avanzado personalice la interfaz para facilitar la interacción (Pressman, 2010).

Ocultar los tecnicismos internos al usuario ocasional: La interfaz de usuario le presentará el mundo virtual de la aplicación. No hay necesidad de preocuparse por el pedido de sistema operativo, funciones de gestión de archivos o cualquier otra tecnología de la información.

Computadora secreta. Básicamente, la interfaz no debe obligar al usuario a interactuar en el nivel "interno" del dispositivo (nunca tendrá que escribir un comando del sistema operativo desde una aplicación de software) (Pressman, 2010).

Diseñar la interacción directa con objetos que aparezcan en la pantalla: El usuario tiene control sobre cuándo puedes manipular los objetos necesarios fin de ejecutar de la misma manera que si el objeto fuera algo físico. Por ejemplo, la interfaz de aplicación que la "extensión" de un objeto (ajuste de su tamaño) permite la implementación de la manipulación directa (Pressman, 2010).

1.2.3.4. Reducir la necesidad de que el usuario memorice

Cuantas más cosas tengan que recordar los usuarios, más probable es que cometan errores al interactuar con el sistema Esta es la razón por la que una interfaz de usuario bien diseñada no sobrecargará la memoria del usuario En la medida de lo posible, el sistema debe

"recordar" la información relevante y ayudar a los usuarios con un guion interactivo que les ayude a recordar. Mandel [Man 97] define los siguientes principios de diseño que permiten que una interfaz reduzca la necesidad de que el usuario recuerde: (Pressman, 2010).

Reducir la demanda de memoria de corto plazo. Cuando los usuarios se involucran en tareas complejas, la demanda de memoria de corto plazo es significativa. La interfaz debe diseñarse para disminuir la necesidad de recordar acciones, entradas y resultados del pasado. Esto se logra dando claves visuales que permitan al usuario reconocer acciones anteriores, en lugar de que tenga que recordarlas (Pressman, 2010).

Hacer que lo preestablecido sea significativo. Lo que al principio se dé por preestablecido debe tener sentido para el usuario promedio, pero éste debería poder especificar sus preferencias individuales. Sin embargo, debe disponerse de la opción de "reiniciar" para restablecer los valores originales (Pressman, 2010).

Definir atajos que sean intuitivos. Cuando se utilice nemotecnia para ejecutar una función del sistema (como la secuencia Ctrl-B para invocar la función de buscar), debe estar ligada con la acción, de modo que sea fácil de recordar (por ejemplo, con la primera letra de la tarea que se va a realizar) (Pressman, 2010).

La distribución visual de la interfaz debe basarse en una metáfora del mundo real. Por ejemplo, un sistema de pagos debe usar una metáfora de chequera y talonario que guíe al usuario a través del proceso de pago. Esto permite que el usuario se base en claves visuales que comprende bien, en vez de tener que memorizar una secuencia críptica de interacciones (Pressman, 2010).

Revelar información de manera progresiva. La interfaz debe estar organizada de manera jerárquica. Es decir, la información acerca de una tarea, objeto o comportamiento debe presentarse primero en un nivel de generalización elevado. Después de que con el ratón el usuario manifieste interés, deben darse más detalles. Un ejemplo, común para muchas aplicaciones de procesamiento de textos, es la función de subrayar. La función en sí es una de varias en el menú estilo del texto. No obstante, no se enlista cada una de las herramientas para subrayar. El usuario debe hacer clic en la opción de subrayar; después se presentan todas las opciones para esta función (una raya, doble raya, línea punteada, etcétera) (Pressman, 2010).

1.2.3.5. *Hacer consistente la interfaz*

La interfaz debe mostrar y recuperar información constantemente. Esto significa: 1) que toda la información se organiza de acuerdo con reglas de diseño que se respetan en todas las pantallas que se muestran, 2) Sus mecanismos de entrada se limitan a un pequeño grupo utilizado de manera coherente en toda la aplicación y 3) mecanismo de cambio de tareas en específico y consistentemente implementado. Mandel [Man97] identifica algunos principios de Diseño de interfaz para que sea consistente: (Pressman, 2010).

Permita que el usuario coloque la tarea en curso en un contexto significativo. Muchas interfaces implementan capas complejas de interacciones con decenas de imágenes en la pantalla. Es importante dar indicadores (títulos en las ventanas, iconos gráficos, código de colores consistente, etc.) que permitan al usuario conocer el contexto del trabajo en curso. Además, debe poder determinar de dónde viene y qué alternativas hay para hacer la transición a una nueva tarea (Pressman, 2010).

Mantener la consistencia en toda la familia de aplicaciones. Todas las aplicaciones (o productos) que hay en un grupo deben implementar las mismas reglas de diseño a fin de que se mantenga la consistencia en toda la interacción (Pressman, 2010).

Si los modelos interactivos anteriores han creado expectativas en el usuario, no haga cambios a menos de que haya una razón ineludible para ello. Una vez que una secuencia interactiva en particular se ha convertido en un estándar (como el uso de alt-G para guardar un archivo), el usuario la espera en toda aplicación que emplea. Un cambio (como utilizar alt-G para invocar la función de modificar la escala) generará confusión. Los principios de diseño de la interfaz analizados en esta sección y en las anteriores dan una guía básica. En las que siguen, el lector aprenderá acerca del proceso de diseño de la interfaz en sí (Pressman, 2010).

1.2.3.6. *Análisis y diseño de la interfaz de usuario.*

El proceso general de análisis y diseño de la interfaz de usuario comienza con la creación de diferentes modelos de comportamiento del sistema (visto desde el exterior). Empezó a definir las tareas requeridas para el usuario y orientadas a la computadora que haga que el sistema funcione, luego considere los aspectos que se aplican a todos los diseños de interfaz. Se emplean las Herramientas utilizadas para la creación de prototipos y la publicación del modelo de diseño y evaluación de la calidad del usuario final (Pressman, 2010).

1.2.3.7. *Análisis de la interfaz.*

Un aspecto clave de todos los modelos de procesos de ingeniería de software es: entender el problema antes de intentar encontrar una solución. En el caso del diseño de la interfaz de usuario, comprender el problema significa comprender: 1) las personas (usuarios finales) que interactuarán con el sistema a través de la interfaz, 2) las tareas que los usuarios finales deben realizar como parte de su trabajo, 3) el contenido se presenta como parte de la interfaz, y 4) el entorno en el que se realizarán estas tareas. En las siguientes secciones, estos elementos del análisis de interfaz se discuten en un intento de sentar una base sólida para las siguientes tareas de diseño (Pressman, 2010).

1.2.3.8. *Análisis del usuario*

Quizás la frase "interfaz de usuario" es toda la justificación para tomarse el tiempo de comprender al usuario antes de preocuparse por los aspectos técnicos. Se dice que cada usuario tiene una imagen mental del programa y esta puede ser diferente a través del cual se entrena a otros usuarios. Además, la imagen mental del usuario puede ser muy diferente del modelo de diseño del ingeniero de software. La única forma en la que se puede hacer converger imágenes mentales y patrones de diseño operativo para comprender a los usuarios y cómo utilizarán el sistema. Para ello, la información de varias fuentes: (Pressman, 2010).

Entrevistas. Este es el enfoque más directo, los miembros del equipo de software se reúnen con los usuarios para comprender mejor sus necesidades, motivaciones, cultura laboral y unos innumerables aspectos adicionales. Esto se hace en persona o a través de grupos de enfoque (Pressman, 2010).

Información sobre ventas. Los representantes de ventas están en contacto regular con los usuarios y reúna información que ayude al equipo del programa a categorizarla y comprenderla mejor sus requerimientos (Pressman, 2010).

Información de mercadotecnia. El análisis de mercado es invaluable para determinar la segmentación del mercado y comprensión de cómo usar el software de alguna manera que se diferencian sutilmente de cada una de estas partes (Pressman, 2010).

Información de apoyo. El equipo de soporte habla diariamente con los usuarios. Crear la fuente de información más probable sobre lo que funciona y lo que no, y lo que les gusta a

los usuarios y lo que no les gusta, tienen preguntas rápidas y fácil de usar. Las siguientes preguntas (adaptadas de [Hac98]) ayudarán a los lectores a comprender mejor a los usuarios del sistema: (Pressman, 2010).

- ¿Los usuarios son profesionales capacitados, técnicos, oficinistas o trabajadores de manufactura?
- ¿Qué nivel de educación formal tiene el usuario promedio?
- ¿Los usuarios son capaces de aprender mediante materiales escritos o han manifestado el deseo de recibir enseñanzas en un aula?
- ¿Los usuarios son mecanógrafos expertos o tienen fobia a los teclados?
- ¿Cuál es el rango de edades de la comunidad de usuarios?
- ¿Los usuarios estarán representados sobre todo por un género?
- ¿Cómo se compensa a los usuarios por el trabajo que realizan?
- ¿Los usuarios trabajan en un horario normal de oficina o hasta terminar el trabajo que hacen?
- ¿El software va a ser parte integral del trabajo de los usuarios o sólo lo emplearán de manera ocasional?
- ¿Cuál es el idioma principal de los usuarios?
- ¿Cuáles son las consecuencias si el usuario comete un error al emplear el sistema?
- ¿Los usuarios son expertos en el tema en el que está centrado el sistema?
- ¿Los usuarios quieren saber sobre la tecnología que hay tras la interfaz?

Una vez respondidas estas preguntas, se sabrá quiénes son los usuarios finales, qué es probable que los motive y agrade, cómo se clasificarían en distintas clases o perfiles de usuarios, cuáles son sus modelos mentales del sistema y cómo debe caracterizarse la interfaz de usuario para que satisfaga sus necesidades (Pressman, 2010).

1.2.3.9. *Etapas del diseño de la interfaz.*

Una vez concluido el análisis de la interfaz, todas las tareas (u objetos y acciones) requeridas por el usuario final habrán sido identificadas en detalle y comenzará la actividad de diseño de la interfaz. El diseño de la interfaz, como todo el de la ingeniería de software, es un proceso iterativo. Cada etapa del diseño de la interfaz de usuario ocurre varias veces, en las que se elabora y refina la información desarrollada en la etapa anterior (Pressman, 2010).

Aunque se han propuesto muchos modelos diferentes para el modelo de la interfaz de usuario ([Nor86], [Nie00]), todos sugieren alguna combinación de las etapas siguientes: (Pressman, 2010).

1. Definir objetos y acciones de la interfaz (operaciones) con el uso de la información desarrollada en el análisis de la interfaz (sección 11.3).
2. Definir eventos (acciones del usuario) que harán que cambie el estado de la interfaz de usuario. Hay que modelar este comportamiento.
3. Ilustrar cada estado de la interfaz como lo vería en la realidad el usuario final.
4. Indicar cómo interpreta el usuario el estado del sistema a partir de la información provista a través de la interfaz (Pressman, 2010).

En ciertos casos, se comienza con bosquejos de cada estado de la interfaz (lo que vería el usuario en diferentes circunstancias) y después se trabaja hacia atrás para definir objetos, acciones y otra información importante del diseño. Sin importar la secuencia de las tareas de diseño, debe 1) siempre apegarse a las reglas doradas estudiadas en la sección 11.1, 2) modelar la forma en la que se va a implementar la interfaz y 3) considerar el ambiente (por ejemplo, tecnología de la pantalla, sistema operativo, herramientas de desarrollo, etc.) que se empleará (Pressman, 2010).

1.2.3.10. *Diseño de una interfaz para WEBAPPS.*

Toda interfaz de usuario —diseñada para una webapp, aplicación de software tradicional, producto de consumo o dispositivo industrial. Dix [Dix99] afirma que debe diseñarse una interfaz de webapp de modo que responda tres preguntas principales del usuario final: (Pressman, 2010).

¿Dónde estoy? La interfaz debe: 1) dar una indicación de la webapp a la que se ha accedido, 2) informar al usuario de su localización en la jerarquía del contenido (Pressman, 2010).

¿Qué puedo hacer ahora? La interfaz siempre debe ayudar al usuario a entender sus opciones actuales: cuáles funciones están disponibles, qué vínculos están vivos, qué contenido es relevante, etcétera (Pressman, 2010).

¿Dónde he estado, hacia dónde voy? La interfaz debe facilitar la navegación. Para ello, debe disponer un “mapa” (implementado en forma tal que sea fácil de entender) que indique dónde ha estado el usuario y las trayectorias que pueden tomarse para moverse a cualquier punto de la webapp (Pressman, 2010).

Una interfaz eficaz de webapp debe dar respuestas a todas estas preguntas cuando el usuario navegue por su contenido y por sus funciones (Pressman, 2010).

1.2.3.11. Principios y lineamientos del diseño de la interfaz

La interfaz de usuario de una webapp es la “primera impresión” que se recibe. Sin importar el valor de su contenido, ni la sofisticación de sus capacidades y servicios de procesamiento, así como el beneficio general de la webapp en sí, una interfaz mal diseñada decepcionará al usuario potencial y en realidad hará que éste vaya a cualquier otro sitio (Pressman, 2010).

Debido al enorme volumen de webapps competidoras en virtualmente toda área temática, la interfaz debe “atrapar” de inmediato al usuario potencial (Pressman, 2010).

Bruce Tognozzi [Tog01] define un conjunto de características fundamentales que todas las interfaces deben tener y con ello establece la filosofía que todo diseñador de interfaces de webapps debe seguir: (Pressman, 2010).

Las interfaces eficaces son atractivas visualmente y perdonan los errores, lo que da a sus usuarios la sensación de tener el control. Los usuarios perciben rápidamente la totalidad de sus opciones, captan cómo lograr sus metas y cómo hacer su trabajo (Pressman, 2010).

Las interfaces eficaces no preocupan al usuario con el funcionamiento interno del sistema. El trabajo se guarda de manera cuidadosa y continua, con opción total para que el usuario deshaga cualquier actividad en cualquier momento (Pressman, 2010).

Las aplicaciones y servicios eficaces realizan un máximo de trabajo, al tiempo que requieren un mínimo de información de parte de los usuarios (Pressman, 2010).

A fin de diseñar interfaces de webapps con estas características, Tognozzi [Tog01] identifica un conjunto de principios generales de diseño: (Pressman, 2010).

Previsión. Una webapp debe diseñarse de modo que prevea el siguiente movimiento del usuario. Por ejemplo, considere una webapp de ayuda al cliente, desarrollada por un

fabricante de impresoras para computadora. Un usuario solicita un objeto de contenido que presenta información sobre el controlador de la impresora para un sistema operativo reciente. El diseñador de la webapp debe prever que el usuario tal vez pida descargar el controlador y debe brindar facilidades de navegación que lo permitan, sin requerir que el usuario busque esta capacidad (Pressman, 2010).

Comunicación. La interfaz debe comunicar el estado de cualquier actividad iniciada por el usuario. La comunicación puede ser obvia (por ejemplo, un mensaje de texto) o sutil (como la imagen de una hoja de papel que se mueva a través de una impresora para indicar que hay una impresión en curso). La interfaz también debe comunicar el estado del usuario (como su identificación) y su ubicación dentro de la jerarquía del contenido de la webapp (Pressman, 2010).

Consistencia. El uso de controles de navegación, menús, iconos y estética (color, forma y distribución) debe ser consistente en la webapp. Por ejemplo, si un texto en color azul y subrayado implica un vínculo de navegación, el contenido nunca debe incorporar texto con dichas características que no impliquen un vínculo. Además, un objeto, digamos un triángulo amarillo, que se utilice para indicar un mensaje de precaución antes de que el usuario invoque una función o acción particular, no debe usarse para otros propósitos en ningún otro lugar de la webapp. Por último, toda característica de la interfaz debe responder de manera consistente con las expectativas del usuario (Pressman, 2010).

Autonomía controlada. La interfaz debe facilitar el movimiento del usuario a través de la webapp, pero lo debe hacer de manera que obligue a respetar las convenciones que se hayan establecido para la aplicación. Por ejemplo, debe controlarse la navegación hacia partes seguras de la webapp por medio de la identificación y clave del usuario, y no debe haber ningún mecanismo de navegación que permita que un usuario evite dichos controles (Pressman, 2010).

Eficiencia. El diseño de la webapp y su interfaz deben optimizar la eficiencia del trabajo del usuario, no la del desarrollador que la diseña y construye ni del ambiente cliente-servidor que la ejecuta. Tognozzi [Tog01] aborda esto cuando escribe lo siguiente: “Esta sencilla verdad explica por qué es tan importante que todos los involucrados en un proyecto de software aprecien la importancia de hacer que la productividad del usuario sea la meta número uno, y de entender la diferencia vital entre construir un sistema eficiente y dar poder a un usuario eficiente.” (Pressman, 2010).

Flexibilidad. La interfaz debe tener flexibilidad suficiente para permitir que algunos usuarios realicen tareas directamente, y que otros exploren la webapp en forma aleatoria (Pressman, 2010).

En cada caso, debe permitir al usuario entender dónde se encuentra y darle la funcionalidad para deshacer errores y volver a trazar trayectorias de navegación mal elegidas (Pressman, 2010).

Centrarse. La interfaz de la webapp (y el contenido que presente) debe mantenerse centrada en las tareas en curso del usuario. En todos los hipermedios, existe la tendencia a llevar al usuario a contenido poco relacionado. ¿Por qué? Porque es muy fácil hacerlo... El problema es que el usuario puede extraviarse con rapidez en muchas capas de información de apoyo y perder de vista el contenido original que buscaba inicialmente (Pressman, 2010).

Ley de Fitt. “El tiempo para llegar a un objetivo está en función de la distancia que hay hasta él y del tamaño que tenga” [Tog01]. Con base en un estudio realizado en la década de 1950 [Fit54], la Ley de Fitt “es un método eficaz para modelar movimientos rápidos e intencionados, donde un apéndice (como una mano) comienza en reposo en una posición específica de arranque y vuelve al reposo dentro de un área objetivo” [Zha02]. Si una secuencia de selecciones o entradas estandarizadas (con muchas opciones diferentes dentro de la secuencia) es definida por una tarea de usuario, la primera selección (con el ratón, por ejemplo) debe estar físicamente cerca de la siguiente selección. Por ejemplo, considere una interfaz de la página inicial de una webapp en un sitio de comercio electrónico que vende productos de consumo electrónicos (Pressman, 2010).

Cada opción del usuario implica un conjunto de elecciones o acciones por seguir. Por ejemplo, la opción “comprar un producto” requiere que el usuario introduzca una categoría de producto seguida por el nombre de éste. La categoría del producto (equipo de audio, televisiones, reproductores DVD, etc.) aparece como menú desplegable tan pronto como se elige “comprar un producto” (Pressman, 2010).

Entonces, la elección siguiente resulta obvia de inmediato (está cerca) y el tiempo para llegar a ella es despreciable. Por el contrario, si la elección apareciera en un menú localizado en el otro lado de la pantalla, el tiempo para que el usuario llegue (y luego elija) será mucho más largo (Pressman, 2010).

Objetos de la interfaz humana. Se ha desarrollado una vasta biblioteca de objetos reutilizables de interfaces humanas para webapps. Úselas. Cualquier objeto de interfaz que

pueda ser “visto, escuchado, tocado o percibido de otro modo” [Tog01] por un usuario final, puede obtenerse de alguna, entre muchas, librerías de objetos (Pressman, 2010).

Reducción de la latencia. En vez de hacer que el usuario espere a que termine alguna operación interna (como descargar una imagen gráfica compleja), la webapp debe usar tareas múltiples, de manera que permita que el usuario continúe con su trabajo mientras finaliza la operación. Además de reducir la latencia, los retrasos deben explicarse de modo que el usuario entienda lo que esté pasando (Pressman, 2010).

Esto incluye: 1) dar retroalimentación auditiva cuando una selección no dé como resultado una acción inmediata por parte de la webapp, 2) desplegar un reloj con animación o una barra de avance que indique que hay un procesamiento en marcha y 3) dar alguna distracción (presentación o texto animado) cuando tenga lugar un procesamiento tardado (Pressman, 2010).

Aprendizaje. Una interfaz de webapp debe diseñarse para minimizar el tiempo de aprendizaje y, una vez aprendida, minimizar el que se dedique a reaprender cuando se regrese a la webapp. En general, la interfaz debe hacer énfasis en un diseño sencillo e intuitivo que organice el contenido y funcionalidad en categorías que resulten obvias para el usuari (Pressman, 2010).

Metáforas. Una interfaz que use una metáfora de interacción es más fácil de aprender y de usar, en la medida en la que la metáfora sea apropiada para la aplicación y el usuario. Una metáfora debe recurrir a imágenes y conceptos salidos de la experiencia del usuario, pero no necesita ser una reproducción exacta de una experiencia del mundo real. Por ejemplo, un sitio de comercio electrónico que implemente el pago automatizado en una institución financiera, usa una metáfora de chequera (lo que no sorprende) para ayudar al usuario a especificar y programar los pagos de sus facturas. Sin embargo, cuando un usuario “escribe” un cheque, no necesita introducir el nombre completo del beneficiario, sino que elige de una lista de ellos o hace que el sistema lo seleccione con base en las primeras letras que escriba.

La metáfora permanece intacta, pero el usuario recibe ayuda de la webapp (Pressman, 2010).

Mantener la integridad de los productos del trabajo. Un producto del trabajo (por ejemplo, un formato llenado por un usuario o una lista especificada por él) debe guardarse en forma automática, de modo que no se pierda si ocurriera un error. Todos hemos experimentado la frustración que surge cuando al terminar de llenar un formato extenso en

una webapp, se pierde su contenido debido a un error (cometido por nosotros, por la webapp o al transmitirlo del cliente al servidor). Para evitar esto, la webapp debe diseñarse para que guarde todos los datos especificados por el usuario. La interfaz debe apoyar esta función y dar al usuario un mecanismo fácil de recuperación de la información “perdida” (Pressman, 2010).

Legibilidad. Toda la información presentada en la interfaz debe ser legible para jóvenes y viejos. El diseñador de la interfaz debe hacer énfasis en estilos legibles para las letras, en su tamaño y en el color del fondo, que debe contrastar (Pressman, 2010).

Dar seguimiento al estado. Cuando resulte apropiado, debe darse seguimiento al estado de la interacción del usuario y guardarlo, de modo que éste pueda salir y volver más tarde para recuperarlo de donde lo haya dejado. En general, las cookies pueden diseñarse para que guarden información del estado. Sin embargo, son una tecnología controvertida y para ciertos usuarios resultan más atractivas otras soluciones de diseño (Pressman, 2010).

Navegación visible. Una interfaz de webapp bien diseñada da “la ilusión de que los usuarios están en el mismo lugar, con el trabajo llevado a ellos” [Tog01]. Cuando se emplea este enfoque, la navegación no es asunto del usuario. En vez de ello, éste recupera objetos del contenido y selecciona funciones que se despliegan y ejecutan a través de la interfaz (Pressman, 2010).

Nielsen y Wagner [Nie96] sugieren algunos lineamientos prácticos para el diseño de interfaces (basados en su rediseño de una webapp importante), que constituyen un buen complemento de los principios ya sugeridos en esta sección: (Pressman, 2010).

- La lectura rápida en un monitor de computadora es aproximadamente 25 por ciento más lenta que la que se hace en un papel. Por tanto, no obligue al usuario a leer grandes cantidades de texto, en particular cuando se explique la operación de la webapp, o dé ayuda para la navegación (Pressman, 2010).
- Evite los avisos “en construcción”: un vínculo innecesario es un camino seguro a la decepción (Pressman, 2010).
- Los usuarios prefieren no desplazar la pantalla. La información importante debe situarse dentro de las dimensiones de una ventana normal de navegación (Pressman, 2010).
- Los menús de navegación y los encabezados deben diseñarse de manera consistente y deben estar disponibles en todas las páginas a las que tenga acceso

el usuario. El diseño no debe basarse en funciones del navegador que ayuden a la navegación (Pressman, 2010).

- La estética nunca debe obstaculizar la funcionalidad. Por ejemplo, un solo botón será una mejor opción de navegación que una imagen agradable pero vaga o que un icono cuyo objetivo no esté claro (Pressman, 2010).
- Las opciones de navegación deben ser obvias, aun para el usuario casual. Éste no debe tener que buscar en la pantalla para determinar cómo entrar a otro contenido o servicios (Pressman, 2010).

Una interfaz bien diseñada mejora la percepción que tenga el usuario del contenido o de los servicios provistos por el sitio. No necesita ser espectacular, pero siempre debe estar bien estructurada y con ergonomía apropiada (Pressman, 2010).

1.2.3.12. Flujo de trabajos para el diseño de la interfaz de webapp

Se dijo que el diseño de la interfaz de usuario comienza con la identificación de los requerimientos del usuario, de la tarea y de los elementos ambientales. Una vez identificadas las tareas del usuario, se crean y analizan los escenarios del usuario (casos de uso) con objeto de definir un conjunto de objetos y acciones de la interfaz (Pressman, 2010).

La información contenida en los formatos del modelo de requerimientos son la base para la creación de una distribución de pantalla que ilustre el diseño gráfico y la ubicación de los iconos, la definición de texto descriptivo en la pantalla, así como la especificación y apilamiento de las ventanas y de los temas mayores y menores del menú. Después se utilizan las herramientas para hacer prototipos e implementar en definitiva el modelo de diseño de la interfaz. Las tareas que siguen representan un flujo de trabajo rudimentario para diseñar una interfaz para webapp: (Pressman, 2010).

1. Revisar la información contenida en el modelo de requerimientos y refinarla según se requiera.

2. Desarrollar un esquema aproximado de la distribución de la interfaz para la webapp.
Como parte de la actividad de modelación de los requerimientos, tal vez se haya desarrollado un prototipo de la interfaz (incluida la distribución). Si ya existe la distribución, debe revisarse y refinarse como se requiera. Si no se hubiera desarrollado la distribución de la interfaz, debe

trabajarse con los participantes para hacerlo en este momento. En la ilustración 17 se presenta una primera distribución esquemática (Pressman, 2010).

3. Mapear los objetivos del usuario en acciones específicas de la interfaz. Para la gran mayoría de webapps, el usuario tendrá un conjunto de objetivos primarios relativamente pequeño. Éstos deben mapearse en acciones específicas de la interfaz, como se muestra en la ilustración 17. En esencia, debe responderse la pregunta siguiente: “¿Cómo hace la interfaz para que el usuario logre cada objetivo?” (Pressman, 2010).

4. Definir un conjunto de tareas de usuario asociadas con cada acción. Cada acción de la interfaz (por ejemplo, “comprar un producto”) se asocia con un conjunto de tareas de usuario. Éstas se identificaron durante la modelación de los requerimientos (Pressman, 2010).

Durante el diseño deben mapearse en interacciones específicas que incluyan los aspectos de navegación, objetos de contenido y funciones de la webapp (Pressman, 2010).

5. Elaborar un guion de las imágenes en la pantalla para cada acción de la interfaz. A medida que se considera cada acción, debe crearse una secuencia de imágenes del guion (imágenes en la pantalla) a fin de ilustrar la manera en la que responde la interfaz a la interacción con el usuario. Deben identificarse los objetos de contenido (aunque todavía no se hayan diseñado ni desarrollado). Deben mostrarse las funciones de la webapp e indicarse los vínculos de navegación (Pressman, 2010).

6. Refinar la distribución de la interfaz y los guiones con entradas del diseño de la estética. En la mayor parte de casos, el lector será el responsable de hacer la distribución aproximada y de elaborar los guiones, pero el aspecto estético y la sensación que genere un sitio comercial importante con frecuencia es desarrollada por un artista, más que por profesionales técnicos. El diseño de la estética se integra con el trabajo realizado por el diseñador de la interfaz (Pressman, 2010).

7. Identificar los objetos de la interfaz de usuario requeridos para implementar la interfaz. Esta tarea quizá requiera buscar en una biblioteca de objetos ya existentes para encontrar aquellos (clases) que sean reutilizables y apropiados para la interfaz de la webapp. Además, en este momento se especifican cualesquiera clases personalizadas (Pressman, 2010).

8. Desarrollar una representación del procedimiento de la interacción del usuario con la interfaz. Esta tarea opcional utiliza diagramas UML de secuencia o diagramas de actividades a fin de ilustrar el flujo de actividades (y decisiones) que tienen lugar cuando el usuario interactúa con la webapp (Pressman, 2010).

9. Desarrollar una representación del comportamiento de la interfaz. Esta opción de tarea emplea diagramas de estado UML (apéndice 1) para representar transiciones de estado y los eventos que las causan. Se definen los mecanismos de control (tales como los objetos y acciones de que dispone el usuario para modificar el estado de una webapp) (Pressman, 2010).

10. Describir la distribución de la interfaz para cada estado. Con el uso de la información de diseño, se asocia una distribución específica o imagen de la pantalla a cada estado de la webapp (Pressman, 2010).

11. Refinar y revisar el modelo del diseño de la interfaz. La revisión de la interfaz debe centrarse en la usabilidad (Pressman, 2010).

Es importante observar que el conjunto definitivo de tareas que se elijan debe adaptarse a los requerimientos especiales de la aplicación que se va a elaborar (Pressman, 2010).

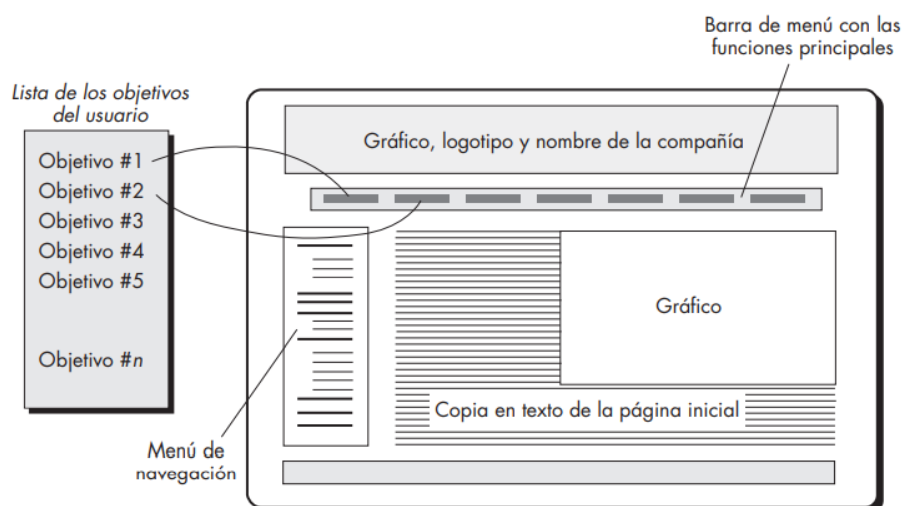


Ilustración 17: Mapeo de los objetivos del usuario en acciones de la interfaz

Fuente: (Pressman, 2010).

1.3. ELEMENTOS DEL DISEÑO DEL DESPLIEGUE

Los elementos de diseño de implementación indican cómo se respaldará la funcionalidad del software y los subsistemas en el entorno informático físico que admitirán. Por ejemplo, los elementos del producto CasaSegura están configurados para funcionar en los tres entornos informáticos principales: la computadora del hogar, la consola de CasaSegura y el servidor operado por CPI Corp. (Permitir el acceso al sistema en línea). Durante el proceso de diseño, se desarrolla un diagrama de implementación y luego se revisa, de la siguiente manera (Pressman, 2010).

Se muestra en la ilustración 18, allí aparecen tres entornos de escritorio (de hecho, habrá más, con sensores y cámaras, entre otras cosas). Los subsistemas (funciones) se encuentran dentro de cada elemento de TI específico. Por ejemplo, una tienda de computadoras personales (Pressman, 2010).

El subsistema que implementa las funciones de gestión familiar, seguimiento y seguridad e información de contacto. Además, un subsistema de acceso externo está diseñado para administrar todos los intentos de acceder al sistema CasaSegura desde una ubicación externa. Cada subsistema se creará para indicar qué componentes implementa. El diagrama de la ilustración 18 es un descriptor de coordenadas. Esto significa que el archivo del diagrama de implementación muestra el entorno informático, pero no indica explícitamente los detalles de configuración. Por ejemplo, no se da ninguna otra definición de "computadora personal". Puede ser un sistema Mac o Windows, una estación de trabajo Sun o un sistema Windows o linux. Estos detalles se publican cuando regresa al diagrama de implementación en formato de versión durante la fase de diseño final o al inicio de la construcción. Se especifica cada versión de la aplicación (una configuración específica, llamada configuración de hardware) (Pressman, 2010).

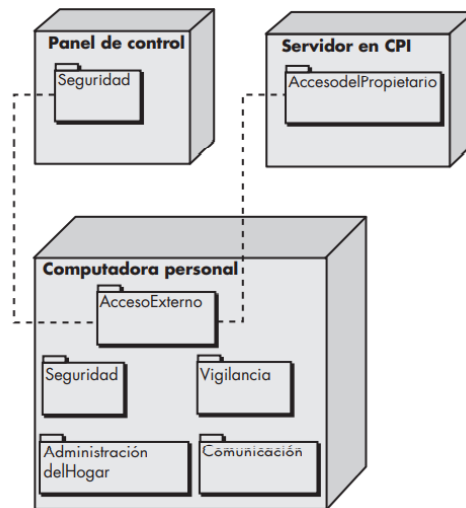


Ilustración 18: Diagrama de despliegue UML

Fuente: (Pressman, 2010).

BIBLIOGRAFÍA

Pressman, R. S. (2010). *Ingeniería del Software Un enfoque práctico* (Séptima Ed). McGraw-Hill. <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>